

Trump Rally Transcripts — NLP Analysis (Colab-ready v2)

How to run

1. *Runtime* → *Change runtime type* → **T4 GPU** (optional, but the transformer steps run several times faster).
2. *Runtime* → **Run all**.
3. When the upload box appears, pick `Trump_Rallies_Dataset.csv`. (If the file is already at `/content/Trump_Rallies_Dataset.csv`, the upload is skipped.)

Set `FAST_MODE = True` in the settings cell for a quicker run with fewer optimisation trials.

0. Install packages

In [1]:

```
%pip install -q bertopic optuna
!python -m spacy download en_core_web_sm
```

```
===== 154.7/154.7 kB 9.0 MB/s eta 0:00:00
===== 442.4/442.4 kB 22.2 MB/s eta 0:00:00
===== 268.7/268.7 kB 23.5 MB/s eta 0:00:00

Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
===== 12.8/12.8 MB 100.3 MB/s eta 0:00:00

✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
△ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

0.1 Imports & settings

In [2]:

```
import os, re, math, time, random, textwrap, warnings
from collections import Counter
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import torch

warnings.filterwarnings("ignore")
os.environ["TOKENIZERS_PARALLELISM"] = "false"

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)

HAS_GPU = torch.cuda.is_available()
PIPE_DEVICE = 0 if HAS_GPU else -1
TORCH_DEVICE = "cuda" if HAS_GPU else "cpu"

# ---- user settings -----
FAST_MODE = False # True -> fewer optimisation trials
```

```

N_TRIALS = 15 if FAST_MODE else 40 # trials per joint-search strategy (Random and TPE)
MIN_TOPICS = 5 # a configuration must produce at least this many topics
MAX_OUTLIER_SHARE = 0.5 # ...and put at most 50% of sentences in the outlier topic (-1)
SENTIMENT_MODEL = "cardiffnlp/twitter-roberta-base-sentiment-latest"
LABEL_MODEL = "google/flan-t5-large" # "google/flan-t5-base" is ~3x smaller/faster
DATA_PATH = "/content/Trump_Rallies_Dataset.csv"
# -----

sns.set_theme(style="whitegrid")
plt.rcParams["figure.dpi"] = 100
print("GPU available:", HAS_GPU)

```

GPU available: True

1. Setup & Data Loading

Loads the 103 rally transcripts (one row per rally: Place, Date, Transcript) and parses the dates so rallies can be ordered chronologically.

In [3]:

```

# upload the csv only if it isn't there yet
if not os.path.exists(DATA_PATH):
    try:
        from google.colab import files
        print("Please upload Trump_Rallies_Dataset.csv")
        uploaded = files.upload()
        fname = next(iter(uploaded))
        if os.path.abspath(fname) != DATA_PATH:
            os.replace(fname, DATA_PATH)
    except ImportError:
        raise FileNotFoundError(f"Place the CSV at {DATA_PATH} or edit DATA_PATH.")

df = pd.read_csv(DATA_PATH)
df = df.drop(columns=[c for c in df.columns if c.startswith("Unnamed")])
df = df.dropna(subset=["Transcript"]).reset_index(drop=True)

df["Date_parsed"] = pd.to_datetime(df["Date"].astype(str).str.strip(), format="%B %d %Y",
errors="coerce")
missing = df["Date_parsed"].isna()
if missing.any():
    df.loc[missing, "Date_parsed"] = pd.to_datetime(df.loc[missing, "Date"], errors="coerce")
df["Place_short"] = (df["Place"].astype(str)
                    .str.replace(r"^Speech:\s*Donald Trump Holds a (Campaign)?Rally in\s*", "",
regex=True)
                    .str.strip())

texts = df["Transcript"].tolist()
print("Shape:", df.shape, "| unparsed dates:", int(df["Date_parsed"].isna().sum()))
display(df[["Place_short", "Date", "Date_parsed"]].head())

```

Shape: (103, 5) | unparsed dates: 0

	Place_short	Date	Date_parsed
0	Sanford, Florida	October 12 2020	2020-10-12
1	Duluth, Minnesota	September 30 2020	2020-09-30
2	Middletown, Pennsylvania	September 26 2020	2020-09-26
3	Newport News, Virginia	September 25 2020	2020-09-25
4	Jacksonville, Florida	September 24 2020	2020-09-24

2. Inspecting a Sample Transcript

In [4]:

```
IDX = 92 if len(df) > 92 else 0
sample = df.loc[IDX, "Transcript"]
print(f"Transcript {IDX} - {df.loc[IDX, 'Place_short']} ({df.loc[IDX, 'Date']}),
{len(sample.split()):,} words\n")
print(textwrap.fill(sample[:3000], 130), "...")
```

Transcript 92 - Speech: Donald Trump Holds a Political Rally in Pensacola, Florida (December 8 2017), 9,454 words

Thank you. Thank you. [Applause, Crowd chants USA] Thank you, everybody. Thank you. And if you think this crowd is big, you should see right now what's outside. Congratulations. [Applause] Hello, Pensacola. What a place. Right here on the Emerald Coast, right? The Emerald Coast in the great state of Florida where we had a tremendous victory, didn't we? Come on. [Applause] We are joined by many, many people in the military and great American patriots. Thank you very much. [Applause] When you remember the campaign, I said, let me begin by wishing each and every one of you a very Merry Christmas. Right? [Applause] And I can think of no better Christmas present for the American people than giving you a massive tax cut. That is what's happening. [Applause] And your Senate and your House hopefully very shortly will have those approvals. Those heads will go up and you will be paying a lot less tax. So, congratulations. I hope. They got to vote and I think they will. I can't wait to sign that tax cut great for so many. But great for the middle class. We're going to put it into law. We're going to cap off an incredible first year in office. There has been no first year like this. Or at least just about -- no, I think the answer is no. But I have to be very accurate because of the fake news back there. [Applause, Crowd Chants Trump] Thank you. In our campaign for the presidency, we defied the pundits, the politicians, and the special interests because ours was a movement of the people and it really was. It was a movement of you. It was a movement of the people. It was a movement to make America great again. It's happening. It's happening. And by the way, how are your 401(k)s doing? Not too bad, right? [Applause] Over the weekend, I was in New York City. And I love to say hello to our folks in uniform whether it's the military, whether it's law enforcement. And we had people backstage and I always want to take my time. And a man comes up. He was a police officer in New York City. The first time anyone said this to me. He said, Mr. President, I want to thank you. My family thinks I'm a financial genius. My 401(k) is up 39 percent in nine months. [Applause] I think it's going to be very hard for somebody to beat us in a few years. Can you imagine now? We are only talking about a few years. All you have to say is, with us it goes up with them, it goes down and that's the end of the election. Right? [Applause] Everyone is doing well and it's going to be a lot better. We are going to do better and better. We are going to make our country great again. And we are looking at our country now as America first. America first. [Crowd Chants USA] So tonight we are going to speak straight to the American people and cut right through the fake news media. Right through. Right through. [Applause] Right through. We're going to speak the plain truth and really the truth that you just want to hear. You have to hear. They don't want to hear. But, and by the way, did you see all ...

3. Named Entity Recognition — Who Gets Mentioned

spaCy `en_core_web_sm` extracts PERSON entities from every transcript (run once and reused in Section 4). Stage-direction artefacts such as "Audience Boos" are filtered out.

In [5]:

```
import spacy
try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    import en_core_web_sm
    nlp = en_core_web_sm.load()
nlp.max_length = 3_000_000

NOISE = re.compile(r"[\[\]\"]|\b|audience\b|bcrowd\b|applause|\bboos?\b|\bbooing\b|chant|inaudible",
re.I)
ner_disable = [p for p in ("tagger", "parser", "attribute_ruler", "lemmatizer") if p in
nlp.pipe_names]

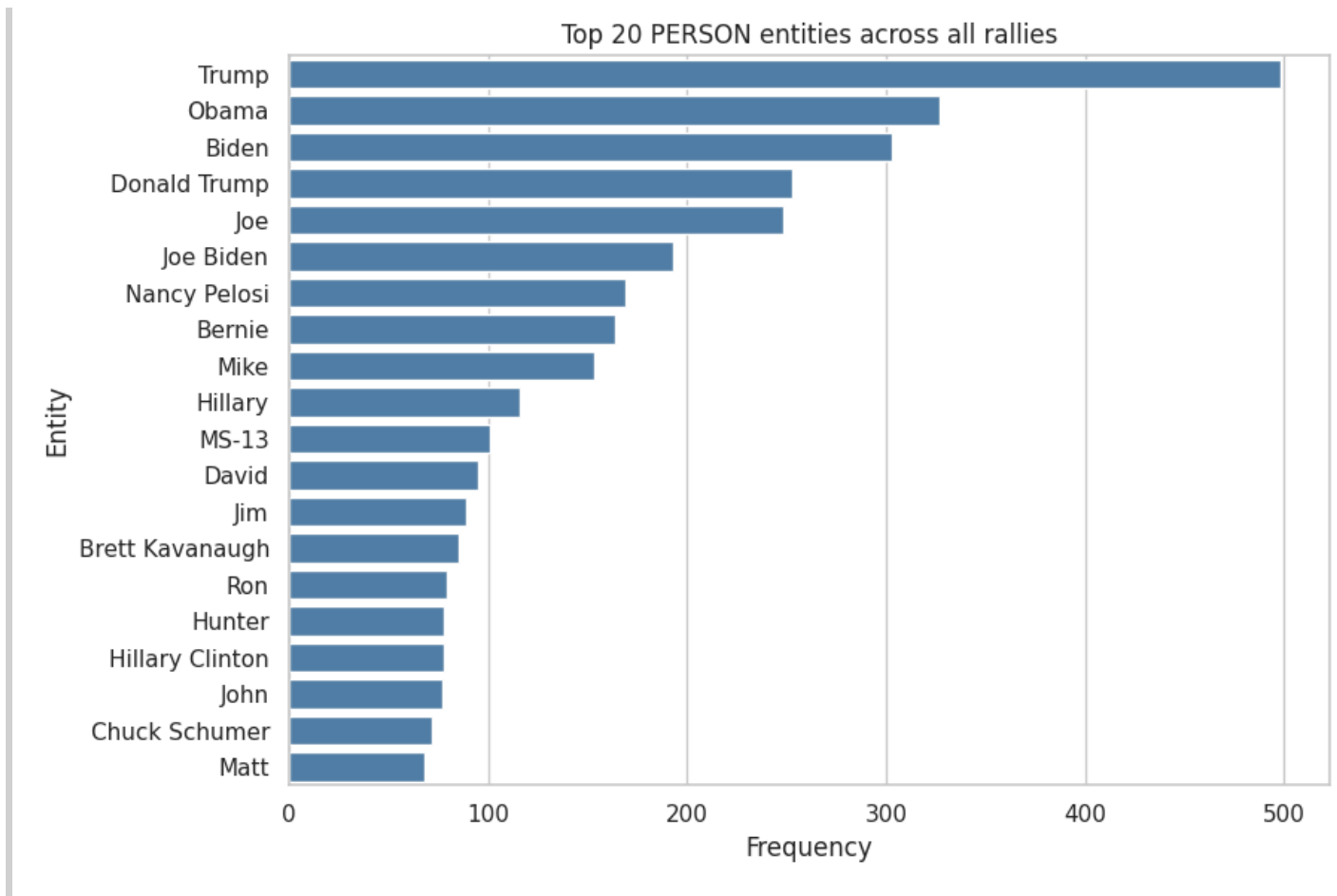
t0 = time.time()
persons_per_doc = []
with nlp.select_pipes(disable=ner_disable):
    for doc in nlp.pipe(texts, batch_size=4):
        persons_per_doc.append([e.text.strip() for e in doc.ents
                                if e.label_ == "PERSON" and not NOISE.search(e.text)])
df["persons"] = persons_per_doc
print(f"NER done in {time.time() - t0:.0f}s")

person_counts = Counter(p for plist in persons_per_doc for p in plist)
top_20_person_entities = pd.DataFrame(person_counts.most_common(20), columns=["Entity", "Frequency"])
display(top_20_person_entities)

plt.figure(figsize=(9, 6))
sns.barplot(data=top_20_person_entities, y="Entity", x="Frequency", color="steelblue")
plt.title("Top 20 PERSON entities across all rallies")
plt.tight_layout(); plt.show()
```

NER done in 82s

	Entity	Frequency
0	Trump	498
1	Obama	327
2	Biden	303
3	Donald Trump	253
4	Joe	248
5	Joe Biden	193
6	Nancy Pelosi	169
7	Bernie	164
8	Mike	153
9	Hillary	116
10	MS-13	101
11	David	95
12	Jim	89
13	Brett Kavanaugh	85
14	Ron	79
15	Hunter	78
16	Hillary Clinton	78
17	John	77
18	Chuck Schumer	72
19	Matt	68



4. Tracking Democratic-Figure Mentions Over Time

Fix: the original cell used an undefined `dates` variable. Here the per-rally counts are aligned with the parsed dates, sorted chronologically and split by person so you can see *who* drives the peaks.

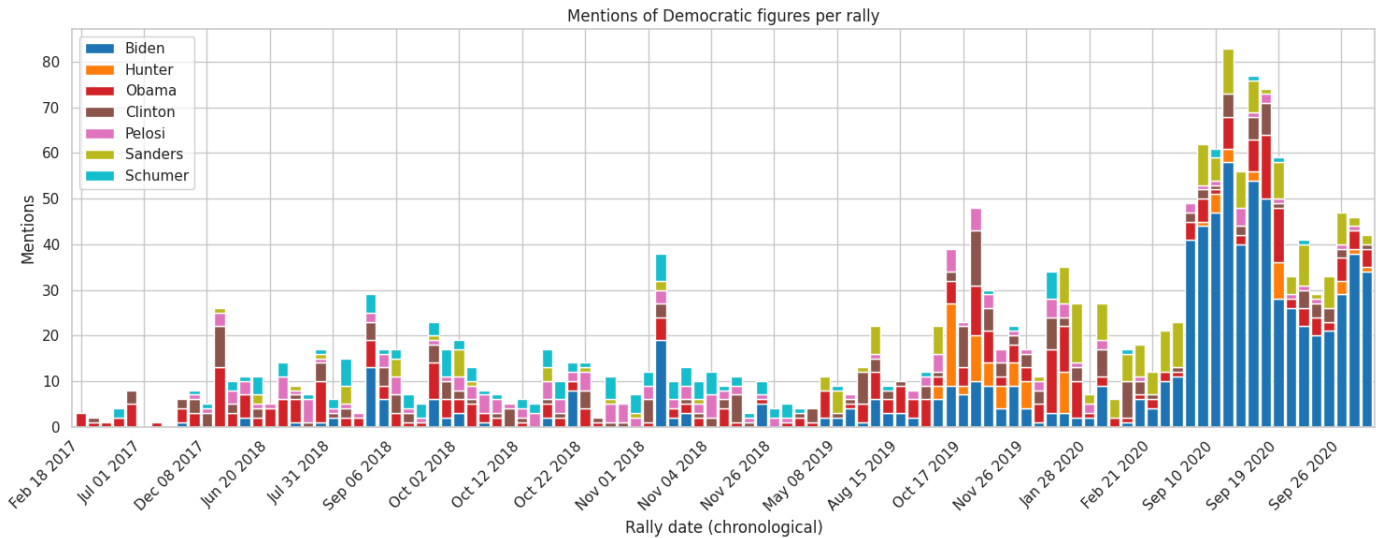
In [6]:

```
GROUPS = {
    "Biden": {"Biden", "Joe", "Joe Biden", "Sleepy Joe", "Sleepy Joe Biden"},
    "Hunter": {"Hunter", "Hunter Biden"},
    "Obama": {"Obama", "Barack Obama"},
    "Clinton": {"Hillary", "Hillary Clinton", "Clinton"},
    "Pelosi": {"Nancy Pelosi", "Pelosi"},
    "Sanders": {"Bernie", "Bernie Sanders"},
    "Schumer": {"Chuck Schumer", "Schumer"},
}
name2group = {n: g for g, names in GROUPS.items() for n in names}

rows = []
for i, plist in enumerate(df["persons"]):
    c = Counter(name2group[p] for p in plist if p in name2group)
    rows.append({"Date": df.loc[i, "Date_parsed"], **{g: c.get(g, 0) for g in GROUPS}})
mentions_df = pd.DataFrame(rows).sort_values("Date").reset_index(drop=True)
mentions_df["Total"] = mentions_df[list(GROUPS)].sum(axis=1)

ax = mentions_df[list(GROUPS)].plot(kind="bar", stacked=True, figsize=(15, 6), width=0.85,
    colormap="tab10")
# only label every 5th date so the axis stays readable
n = 5
ax.set_xticks(range(0, len(mentions_df), n))
ax.set_xticklabels(mentions_df["Date"].dt.strftime("%b %d %Y").iloc[::n], rotation=45, ha="right")
ax.set_xlabel("Rally date (chronological)"); ax.set_ylabel("Mentions")
```

```
ax.set_title("Mentions of Democratic figures per rally")
plt.tight_layout(); plt.show()
```



5. Transcript Similarity (TF-IDF + Cosine Similarity)

Pairwise cosine similarity between all rallies — bright blocks mean reused material. The table lists the most similar pairs.

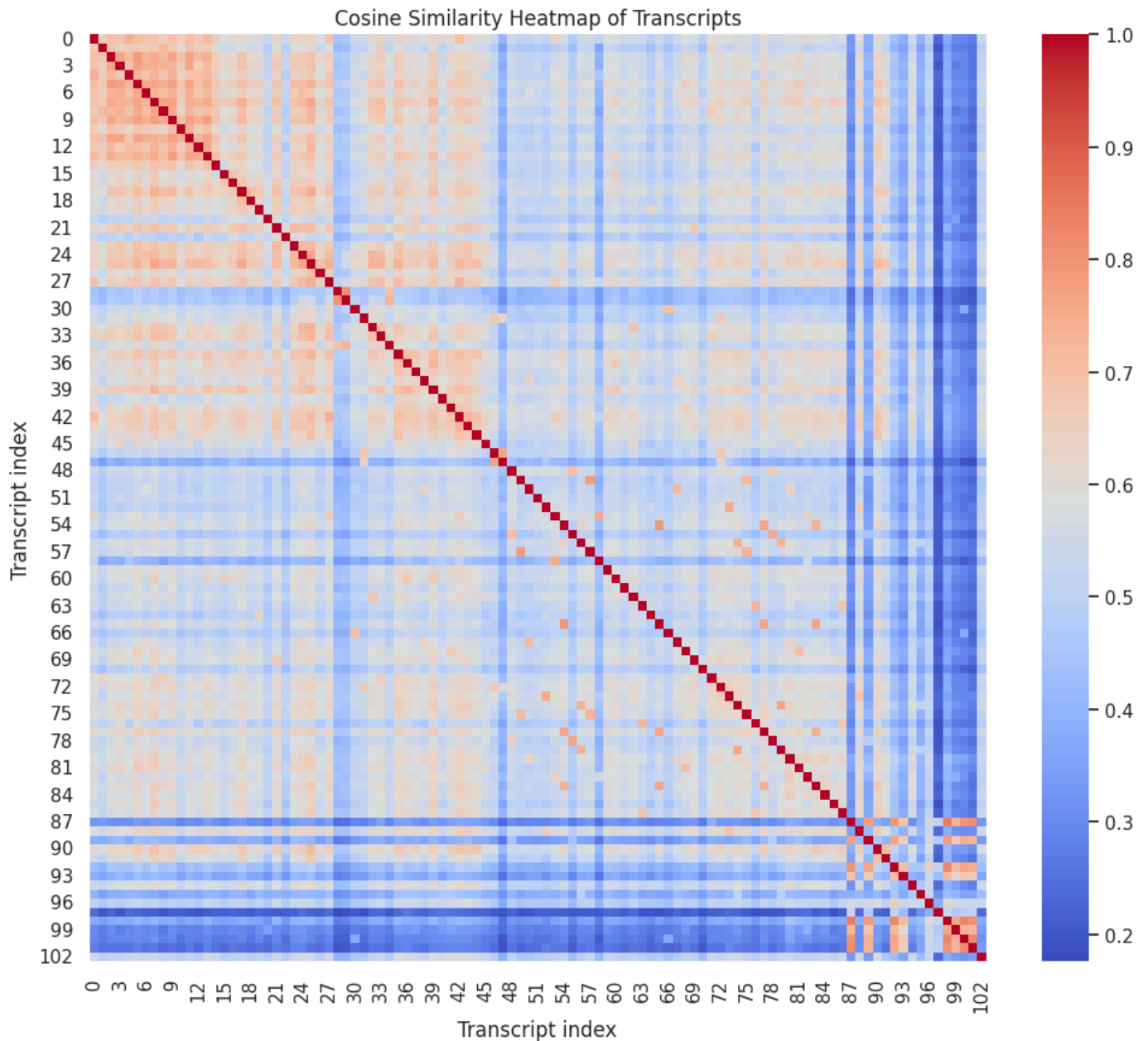
In [7]:

```
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

tfidf_matrix = TfidfVectorizer(stop_words="english").fit_transform(texts)
sim = cosine_similarity(tfidf_matrix)

plt.figure(figsize=(12, 10))
sns.heatmap(sim, cmap="coolwarm")
plt.title("Cosine Similarity Heatmap of Transcripts")
plt.xlabel("Transcript index"); plt.ylabel("Transcript index")
plt.show()

iu = np.triu_indices_from(sim, k=1)
order = np.argsort(sim[iu][:,-1][:8])
pairs = pd.DataFrame({"A": iu[0][order], "B": iu[1][order], "cosine": sim[iu][order].round(3)})
pairs["A rally"] = pairs["A"].map(lambda i: f"{df.Place_short[i][:40]} ({df.Date[i]})")
pairs["B rally"] = pairs["B"].map(lambda i: f"{df.Place_short[i][:40]} ({df.Date[i]})")
display(pairs)
```



	A	B	cosine	A rally	B rally
0	87	98	0.844	Speech: Donald Trump Holds a Political R (June...	Speech: Donald Trump Holds a Political R (June...
1	98	100	0.827	Speech: Donald Trump Holds a Political R (June...	Speech: Donald Trump Holds a Political R (Marc...
2	92	98	0.827	Speech: Donald Trump Holds a Political R (Dece...	Speech: Donald Trump Holds a Political R (June...
3	87	101	0.817	Speech: Donald Trump Holds a Political R (June...	Speech: Donald Trump Holds a Political R (Marc...
4	100	101	0.811	Speech: Donald Trump Holds a Political R (Marc...	Speech: Donald Trump Holds a Political R (Marc...
5	98	101	0.807	Speech: Donald Trump Holds a Political R (June...	Speech: Donald Trump Holds a Political R (Marc...
6	28	29	0.801	Speech: Donald Trump Holds a Political R (Nove...	Speech: Donald Trump Holds a Political R (Nove...
7	87	100	0.798	Speech: Donald Trump Holds a Political R (June...	Speech: Donald Trump Holds a Political R (Marc...

6. Extracting Audience / Crowd Reactions

Every bracketed cue is captured with the text before it, then **non-crowd tags such as [Inaudible] are removed**. The *Preceding Phrase* is the last two sentences spoken before the reaction (the original kept the whole chunk since the previous tag, which can run to hundreds of words).

In [8]:

```
TAG_RE = re.compile(r"\[[^\]]+\]")
CROWD_RE = re.compile(r"audience|crowd|applause|cheer|\bboo|chant|laugh|clap|yell|shout|scream", re.I)
SENT_SPLIT = re.compile(r"(?<=[.!?])\s+")

# pair each [tag] with the text that came right before it
def extract_reactions(transcript):
    out, last = [], 0
    for m in TAG_RE.finditer(transcript):
        out.append((transcript[last:m.start()].strip(), m.group(0).strip()))
        last = m.end()
    return out

def last_sentences(text, k=2):
    sents = [s.strip() for s in SENT_SPLIT.split(TAG_RE.sub(" ", text)) if len(s.strip()) > 1]
    return re.sub(r"\s+", " ", " ".join(sents[-k:])).strip()

def reaction_type(tag):
    t = tag.lower()
    if "boo" in t: return "Boos"
    if "laugh" in t: return "Laughter"
    if "chant" in t: return "Chant"
    if any(x in t for x in ("applause", "cheer", "clap")): return "Applause/Cheers"
    return "Other crowd"

rows = []
for i, tr in enumerate(texts):
    for prec, tag in extract_reactions(tr):
        rows.append({"doc_id": i, "Date": df.loc[i, "Date_parsed"], "Preceding Text": prec, "Crowd
Reaction": tag})
reaction_all = pd.DataFrame(rows)

reaction_df = reaction_all[reaction_all["Crowd Reaction"].str.contains(CROWD_RE)].copy()
reaction_df["Preceding Phrase"] = reaction_df["Preceding Text"].apply(last_sentences)
reaction_df = reaction_df[reaction_df["Preceding Phrase"].str.split().str.len() >=
3].reset_index(drop=True)
reaction_df["Reaction Type"] = reaction_df["Crowd Reaction"].apply(reaction_type)

print(f"All bracketed tags: {len(reaction_all):,} | crowd reactions kept: {len(reaction_df):,}")
print(reaction_df["Reaction Type"].value_counts())
display(reaction_df[["Preceding Phrase", "Crowd Reaction", "Reaction Type"]].head())
```

All bracketed tags: 2,725 | crowd reactions kept: 2,501

Reaction Type
Applause/Cheers 1006
Boos 690
Chant 403
Laughter 239
Other crowd 163
Name: count, dtype: int64

	Preceding Phrase	Crowd Reaction	Reaction Type
0	Thank you. It's great to be back.	[Audience chants "USA"]	Chant
1	It's you know, here we are. But we're going to...	[Audience chants "We love you"]	Chant
2	We're going to win four more years in the Whit...	[Audience chants "USA"]	Chant
3	Most important we've ever had. Biden has made ...	[Audience Boos]	Boos
4	Don't blow it, don't blow it. Sleepy Joe wants...	[Audience Boos]	Boos

7. Sentiment of Crowd Reactions — SentiWordNet (original method)

Kept for comparison. Scoring the words inside a tag like [Audience Boos] with a dictionary is a weak signal, which is why Section 7b adds a transformer model applied to what was actually said.

In [9]:

```
import nltk
for pkg in ("sentiwordnet", "wordnet", "omw-1.4", "punkt", "punkt_tab"):
    nltk.download(pkg, quiet=True)
from nltk.corpus import sentiwordnet as swn
from nltk.tokenize import word_tokenize

def analyze_sentiment_swn(text):
    score, count = 0.0, 0
    for tok in word_tokenize(re.sub(r"\\[\\]\\\"", " ", text)):
        syns = list(swn.senti_synsets(tok.lower()))
        if syns:
            score += syns[0].pos_score() - syns[0].neg_score()
            count += 1
    if count == 0:
        return np.nan
    avg = score / count
    return 1 if avg > 0 else (0 if avg < 0 else np.nan)

reaction_df["SWN Label"] = reaction_df["Crowd Reaction"].apply(analyze_sentiment_swn)
print("SentiWordNet label by reaction type (NaN = neutral/unscored):")
display(pd.crosstab(reaction_df["Reaction Type"], reaction_df["SWN Label"].map({1: "positive", 0: "negative"}).fillna("neutral")))
```

SentiWordNet label by reaction type (NaN = neutral/unscored):

SWN Label	negative	neutral	positive
Reaction Type			
Applause/Cheers	1	58	947
Boos	545	145	0
Chant	9	363	31
Laughter	0	239	0
Other crowd	32	97	34

7b. Transformer Sentiment of the Speech (Twitter-RoBERTa)

Labels each *Preceding Phrase* as negative / neutral / positive with `cardiffnlp/twitter-roberta-base-sentiment-latest`, then compares that with how the crowd reacted. This replaces the external `filtered_phrases_nouns_verbs.xlsx`: the same kind of table (`Filtered_phrase` = nouns, proper nouns and verbs of the phrase, `label` = 1 positive / 0 negative) is rebuilt here from the original transcripts.

In [10]:

```
from transformers import pipeline

sent_pipe = pipeline("text-classification", model=SENTIMENT_MODEL, tokenizer=SENTIMENT_MODEL,
device=PIPE_DEVICE)

def roberta_sentiment(text_list, batch_size=32):
    out = sent_pipe(list(text_list), batch_size=batch_size, truncation=True, max_length=512)
    return [o["label"].lower() for o in out], [float(o["score"]) for o in out]
```

```

t0 = time.time()
reaction_df["roberta_label"], reaction_df["roberta_score"] = roberta_sentiment(reaction_df["Preceding
Phrase"])
print(f"Scored {len(reaction_df):,} phrases in {time.time() - t0:.0f}s")

order_lab = ["negative", "neutral", "positive"]
fig, axes = plt.subplots(1, 2, figsize=(15, 4.5), gridspec_kw={"width_ratios": [1, 2]})
reaction_df["roberta_label"].value_counts().reindex(order_lab).plot(
    kind="bar", ax=axes[0], color=["tab:red", "tab:gray", "tab:green"])
axes[0].set_title("Speech sentiment before crowd reactions"); axes[0].set_xlabel("")
ct = pd.crosstab(reaction_df["Reaction Type"], reaction_df["roberta_label"],
    normalize="index").reindex(columns=order_lab)
sns.heatmap(ct, annot=True, fmt=".0%", cmap="RdYlGn", ax=axes[1], vmin=0, vmax=1)
axes[1].set_title("Share of speech sentiment per crowd reaction type"); axes[1].set_xlabel("speech
sentiment")
plt.tight_layout(); plt.show()

# Rebuild the 'filtered phrases (nouns & verbs)' dataset from the original transcripts
KEEP_NV = {"NOUN", "PROPN", "VERB"}
filtered = []
for doc in nlp.pipe(reaction_df["Preceding Phrase"].tolist(), batch_size=64):
    filtered.append(" ".join(t.lower_ for t in doc
        if t.is_alpha and t.pos_ in KEEP_NV and not t.is_stop and len(t) > 1))
reaction_df["Filtered_phrase"] = filtered

df1 = reaction_df[reaction_df["roberta_label"] != "neutral"].copy()
df1["label"] = (df1["roberta_label"] == "positive").astype(int)
df1 = df1[df1["Filtered_phrase"].str.len() > 0].reset_index(drop=True)
print("Labelled phrase set:", df1.shape, "| positive share:", round(df1["label"].mean(), 3))
display(df1[["Preceding Phrase", "Filtered_phrase", "label", "Reaction Type"]].head())

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

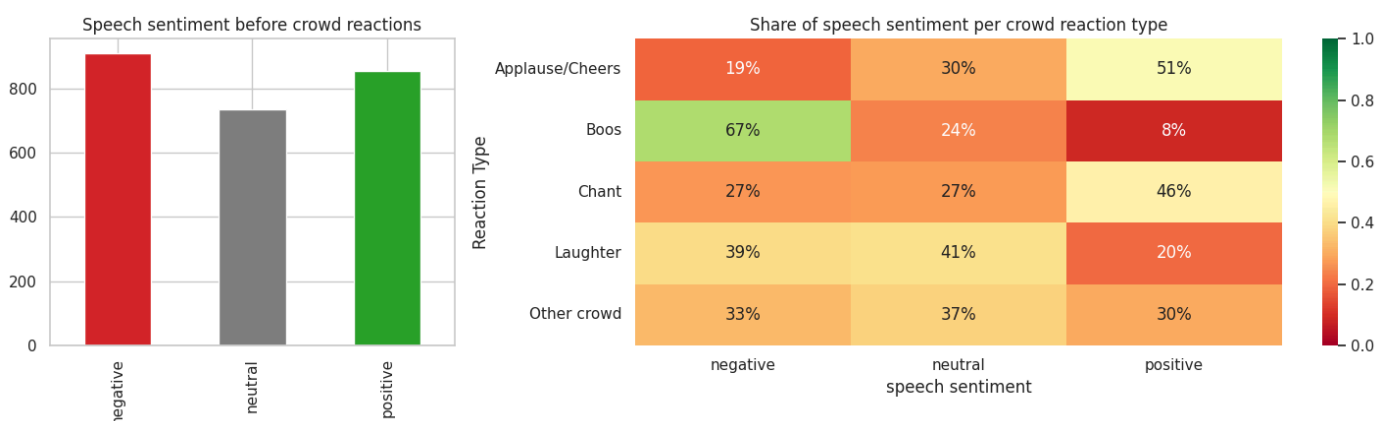
[transformers] RobertaForSequenceClassification LOAD REPORT from: cardiffnlp/twitter-roberta-base-sentiment-latest

Key	Status		
roberta.pooler.dense.bias	UNEXPECTED		
roberta.pooler.dense.weight	UNEXPECTED		

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Scored 2,501 phrases in 9s



Labelled phrase set: (1736, 11) | positive share: 0.487

	Preceding Phrase	Filtered_phrase	label	Reaction Type
0	Thank you. It's great to be back.	thank	1	Chant

	Preceding Phrase	Filtered_phrase	label	Reaction Type
1	It's you know, here we are. But we're going to...	know going finish going country	1	Chant
2	We're going to win four more years in the Whit...	going win years white house going country	1	Chant
3	Most important we've ever had. Biden has made ...	biden bargain exchange party nomination handed...	0	Boos
4	Don't blow it, don't blow it. Sleepy Joe wants...	blow blow sleepy joe wants quadruple taxes kno...	0	Boos

8. Feature Importance for Sentiment — Mutual Information

Words are encoded as binary presence (the right input for discrete mutual information). A word counts as "positive" if phrases containing it are positive more often than the overall base rate — computed with exact token matches (the original `str.contains` matched substrings, so `ve` matched almost everything).

In [11]:

```
from sklearn.feature_selection import mutual_info_classif, chi2

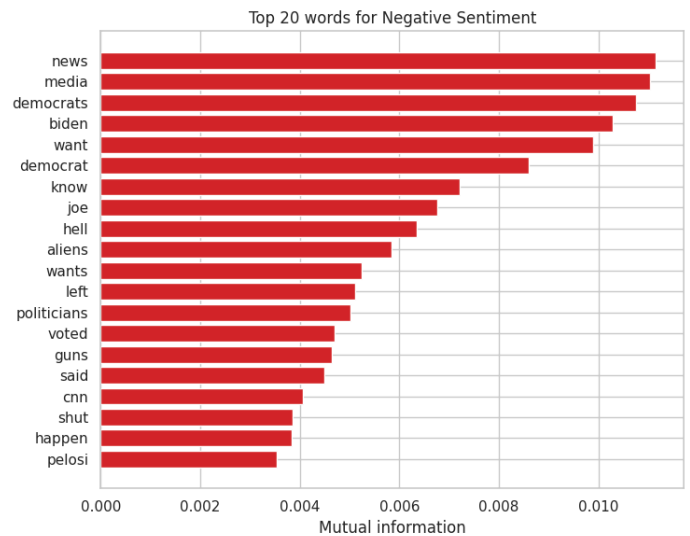
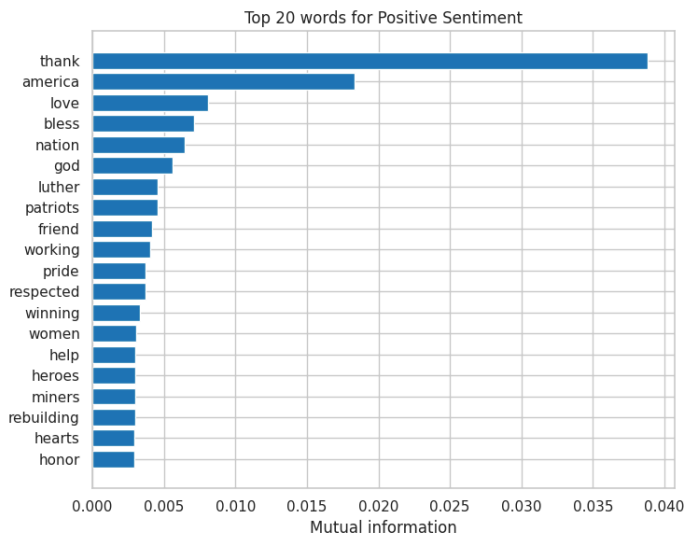
# share of positive phrases each word shows up in
def word_direction(X, y):
    y = np.asarray(y, dtype=float)
    doc_freq = np.asarray(X.sum(axis=0)).ravel()
    pos = np.asarray(X.T @ y).ravel()
    return pos / np.maximum(doc_freq, 1)

def plot_two(pos, neg, title_pos, title_neg, xlabel):
    h = max(6, 0.28 * max(len(pos), len(neg)))
    fig, axes = plt.subplots(1, 2, figsize=(15, h))
    for ax, feats, color, title in ((axes[0], pos, "tab:blue", title_pos), (axes[1], neg, "tab:red",
title_neg)):
        if feats:
            w, s = zip(*feats)
            ax.barh(w, s, color=color)
            ax.invert_yaxis()
            ax.set_title(title); ax.set_xlabel(xlabel)
    plt.tight_layout(); plt.show()

y = df1["label"].values
base_rate = y.mean()

cv_bin = CountVectorizer(binary=True, max_features=1000, min_df=3)
X = cv_bin.fit_transform(df1["Filtered_phrase"])
names = cv_bin.get_feature_names_out()
mi = mutual_info_classif(X, y, discrete_features=True, random_state=SEED)
direction = word_direction(X, y)

top_pos = sorted([(n, s) for n, s, d in zip(names, mi, direction) if d > base_rate], key=lambda x:
x[1], reverse=True)[:20]
top_neg = sorted([(n, s) for n, s, d in zip(names, mi, direction) if d < base_rate], key=lambda x:
x[1], reverse=True)[:20]
plot_two(top_pos, top_neg, "Top 20 words for Positive Sentiment", "Top 20 words for Negative
Sentiment", "Mutual information")
```



9. Feature Importance for Sentiment — Chi-Square Test

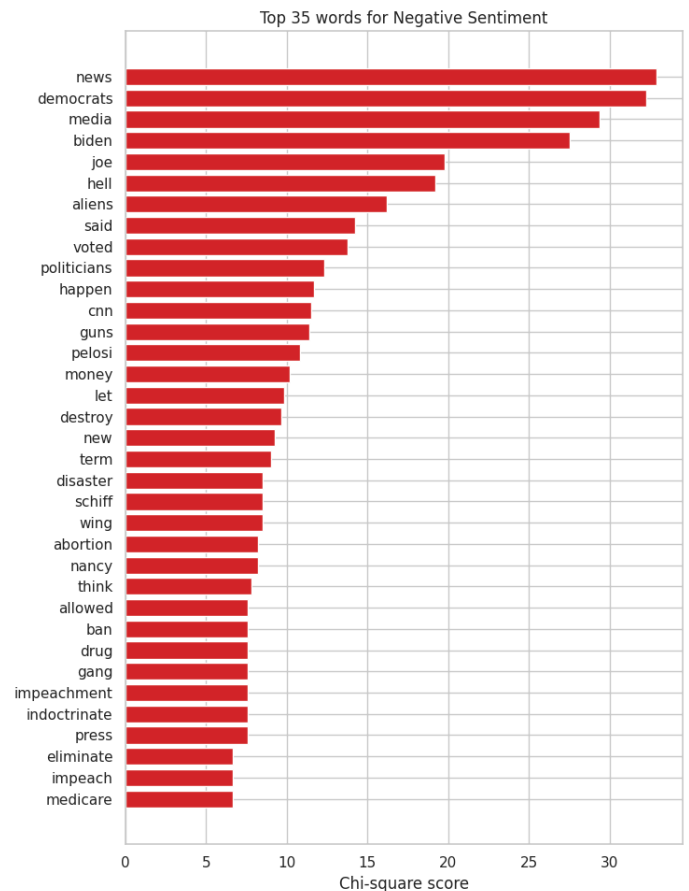
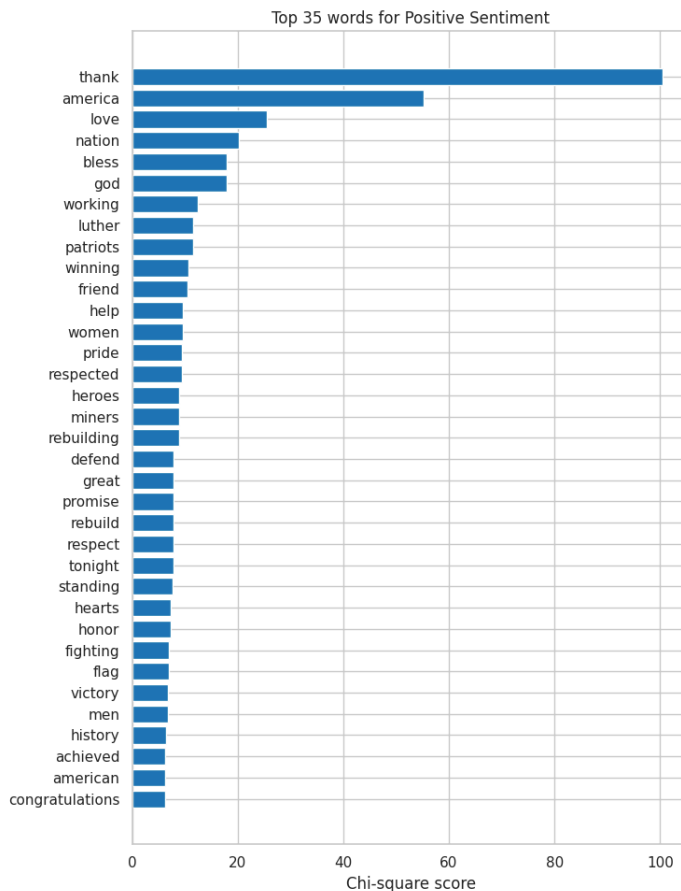
Same idea with the original domain-specific stopwords list removed first. (Fix: `chi2` is now imported.)

In [12]:

```
DOMAIN_STOP = {'know', 'want', 'wants', 'supports', 'crooked', 'party', 'straight', 'saw', 'moment',
               'left', 'guys', 'happened', 'idea', 'leader', 'lou', 'lies', 'gets', 'democrat',
               'trying', 'made', 'keep', 'need', 'telling', 'call', 'shut'}

phr = df1["Filtered_phrase"].apply(lambda p: " ".join(w for w in p.split() if w not in DOMAIN_STOP))
cv_chi = CountVectorizer(binary=True, max_features=1000, min_df=3)
X2 = cv_chi.fit_transform(phr)
names2 = cv_chi.get_feature_names_out()
chi2_scores, p_values = chi2(X2, y)
direction2 = word_direction(X2, y)

top_pos = sorted([(n, s) for n, s, d in zip(names2, chi2_scores, direction2) if d > base_rate],
                  key=lambda x: x[1], reverse=True)[:35]
top_neg = sorted([(n, s) for n, s, d in zip(names2, chi2_scores, direction2) if d < base_rate],
                  key=lambda x: x[1], reverse=True)[:35]
plot_two(top_pos, top_neg, "Top 35 words for Positive Sentiment", "Top 35 words for Negative Sentiment", "Chi-square score")
```



10. Extracting Biden-Related Sentences

Keeps every sentence mentioning "Joe" or "Biden" (bracketed stage directions removed), along with the rally date for the topics-over-time plot. Note that "Joe" can occasionally refer to someone else.

In [13]:

```
BIDEN_RE = re.compile(r"\b(joe|biden)\b", re.I)
all_biden_sentences, biden_dates = [], []
for i, tr in enumerate(texts):
    for s in SENT_SPLIT.split(TAG_RE.sub(" ", tr)):
        s = re.sub(r"\s+", " ", s).strip()
        if BIDEN_RE.search(s):
            all_biden_sentences.append(s)
            biden_dates.append(df.loc[i, "Date_parsed"])

print(len(all_biden_sentences), "sentences\n")
print("\n".join(all_biden_sentences[:10]))
```

878 sentences

You know our competitor, sleepy Joe, he had a rally today and practically nobody showed up.
 Sleepy Joe, but it's great to be back in my home state, Florida to make my official return to the campaign trail.
 Sleepy Joe Biden, not a nice guy, by the way, I have to be honest.
 No, Biden had a bad day.
 Now Biden had a bad day.
 Joe Biden is also owned by the radical globalists, the wealthy donors, the big money special interests who shipped away your jobs, shut down your factories, threw open your borders, and ravaged your cities while sacrificing American blood and treasure in this ridiculous -- endless wars, set of endless wars that we've been in for a long time, 19 years.
 Biden has made a corrupt bargain exchange for his party's nomination, he's handed control to the Socialists, the Marxists, and the left-wing extremists and you know that and he's got no strength left, he's got no power left.

Sleepy Joe wants to quadruple your taxes, you know, wants to quadruple.
And by the way, Obama and Biden knew everything that was happening, OK?
Joe Biden and the Democrat Socialist, but that gives you another reason to go out and vote.

11. Topic Modeling Preparation

- **Cleaning** (original rules): keep nouns/verbs/adjectives/adverbs, drop the custom stopwords. Sentences with fewer than two words left are dropped — these produced the empty topic (5____) and the degenerate `vote vote vote / sleepy sleepy` clusters in the original run.
- **Coherence (c_v)** is computed against a reference corpus of all full transcripts, with the standard c_v definition (boolean sliding window of 110 tokens, NPMI, indirect cosine). It is implemented directly in NumPy so no `gensim` install is needed (`gensim` often forces a NumPy downgrade and a runtime restart on Colab). `c_npmi` (window 10) is reported as a second opinion.

In [14]:

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

CUSTOM_STOP = {
    "said", "think", "got", "let", "like", "did", "going", "remember", "know",
    "compared", "months", "day", "thing", "people", "bad", "right", "can", "joe",
    "biden", "had", "now", "say", "go", "gon", "just", "have", "here", "so", "do", "get", "as",
    "need", "done", "then"}
KEEP_POS = {"NOUN", "VERB", "ADJ", "ADV"}

cleaned = []
for doc in nlp.pipe(all_biden_sentences, batch_size=64):
    cleaned.append(" ".join(t.lower_ for t in doc
                            if t.is_alpha and t.pos_ in KEEP_POS and t.lower_ not in CUSTOM_STOP and
                            not t.is_stop))

keep = [i for i, c in enumerate(cleaned) if len(c.split()) >= 2]
DOCS = [cleaned[i] for i in keep] # text used for topic words (c-TF-IDF)
DOCS_ORIG = [all_biden_sentences[i] for i in keep] # original sentences (embeddings, examples)
DOCS_DATES = [biden_dates[i] for i in keep]
print(f"{len(DOCS)} of {len(cleaned)} sentences kept for topic modeling")
```

714 of 878 sentences kept for topic modeling

In [15]:

```
class Coherence:
    # c_v / c_npmi topic coherence over a tokenised reference corpus (boolean sliding windows).
    def __init__(self, token_docs, window, stride=1, cache_size=2500):
        vocab, ids, starts, ends, offset = {}, [], [], [], 0
        for toks in token_docs:
            L = len(toks)
            if L == 0:
                continue
            ids.extend(vocab.setdefault(w, len(vocab)) for w in toks)
            s = np.array([0]) if L <= window else np.arange(0, L - window + 1, stride)
            starts.append(s + offset)
            ends.append(np.minimum(s + window, L) + offset)
            offset += L
        ids = np.asarray(ids)
        self.order = np.argsort(ids, kind="stable") # token positions grouped by word id
        self.bounds = np.searchsorted(ids[self.order], np.arange(len(vocab) + 1))
        self.vocab, self.starts, self.ends = vocab, np.concatenate(starts), np.concatenate(ends)
        self.n = len(self.starts)
        self._cache, self._cache_size = {}, cache_size

    def presence(self, w):
```

```

    if w in self._cache:
        return self._cache[w]
    wid = self.vocab.get(w)
    v = None
    if wid is not None:
        pos = self.order[self.bounds[wid]:self.bounds[wid + 1]]
        v = (np.searchsorted(pos, self.ends) - np.searchsorted(pos, self.starts)) > 0
    if len(self._cache) < self._cache_size:
        self._cache[w] = v
    return v

def score_topic(self, words, measure="c_v"):
    P = [self.presence(w) for w in words]
    P = [p for p in P if p is not None and p.any()]
    if len(P) < 2:
        return np.nan
    M = np.vstack(P).astype(np.float32)
    p = M.mean(axis=1)
    joint = (M @ M.T) / self.n
    eps = 1e-12
    npmi = np.log((joint + eps) / np.outer(p, p)) / -np.log(joint + eps)
    if measure == "c_v":
        s = npmi.sum(axis=0)
        cos = (npmi @ s) / (np.linalg.norm(npmi, axis=1) * np.linalg.norm(s) + eps)
        return float(cos.mean())
    iu = np.triu_indices(len(P), k=1)
    return float(npmi[iu].mean())

REF_STOP = set(ENGLISH_STOP_WORDS) | CUSTOM_STOP
def ref_tokens(text):
    text = TAG_RE.sub(" ", text).lower()
    return [w for w in re.findall(r"[a-z]+", text) if w not in REF_STOP and len(w) > 1]

ref_docs = [ref_tokens(t) for t in texts]
CV = Coherence(ref_docs, window=110, stride=10) # c_v (stride 10 keeps memory low; near-identical to stride 1)
NP = Coherence(ref_docs, window=10, stride=5) # c_npmi
print(f"Reference corpus: {sum(map(len, ref_docs))}, tokens | c_v windows: {CV.n:,} | npmi windows: {NP.n:,}")
print("Sanity check c_v(tax, taxes, cut, cuts, income) =", round(CV.score_topic(["tax", "taxes", "cut", "cuts", "income"]), 3))
print("c_v(tax, military, beautiful, china, vote) =", round(CV.score_topic(["tax", "military", "beautiful", "china", "vote"]), 3))

```

```

Reference corpus: 349,167 tokens | c_v windows: 33,837 | npmi windows: 69,683
Sanity check c_v(tax, taxes, cut, cuts, income) = 0.818
c_v(tax, military, beautiful, china, vote) = 0.382

```

11.1 Sentence embeddings (computed once, reused by every experiment)

Three free sentence-transformer models × two inputs (original sentence vs. cleaned text). The original notebook embedded the cleaned text; transformers usually do better on the full sentence, so both are tested.

In [16]:

```

from sentence_transformers import SentenceTransformer

EMB_MODELS = {
    "mpnet": "sentence-transformers/all-mpnet-base-v2",
    "minilm": "sentence-transformers/all-MiniLM-L6-v2",
    "bge-small": "BAAI/bge-small-en-v1.5",
}
ST, EMB = {}, {}
t0 = time.time()

```

```

for short, name in EMB_MODELS.items():
    ST[short] = SentenceTransformer(name, device=TORCH_DEVICE)
    for inp, data in (("raw", DOCS_ORIG), ("cleaned", DOCS)):
        EMB[f"{short}|{inp}"] = ST[short].encode(data, batch_size=64, show_progress_bar=False,
                                                normalize_embeddings=True)
print(f"{len(EMB)} embedding sets in {time.time() - t0:.0f}s:", list(EMB))

```

```

6 embedding sets in 17s: ['mpnet|raw', 'mpnet|cleaned', 'minilm|raw', 'minilm|cleaned', 'bge-small|raw', 'bge-
small|cleaned']

```

11.2 BERTopic experiment helpers

Every fit is logged with its configuration, `c_v`, `c_npmi`, topic diversity, number of topics and outlier share. A configuration is **valid** only if it yields at least `MIN_TOPICS` topics and at most `MAX_OUTLIER_SHARE` outliers — otherwise an optimiser could "win" by discarding most sentences or collapsing to 2 topics.

In [17]:

```

from bertopic import BERTopic
from bertopic.vectorizers import ClassTfidfTransformer
from umap import UMAP
from hdbscan import HDBSCAN

BASE_CFG = dict(emb="mpnet|raw", n_neighbors=15, n_components=5, min_cluster_size=10,
                min_samples=None,
                method="eom", ngram_max=1, reduce_frequent_words=False, bm25=False, nr_topics=None)

LOG = []

def build_model(c, seed=SEED, representation_model=None, embedding_model=None):
    ms = c["min_samples"]
    return BERTopic(
        embedding_model=embedding_model,
        umap_model=UMAP(n_neighbors=int(c["n_neighbors"]), n_components=int(c["n_components"]),
                        min_dist=0.0, metric="cosine", random_state=seed),
        hdbscan_model=HDBSCAN(min_cluster_size=int(c["min_cluster_size"]),
                              min_samples=None if ms is None else int(ms),
                              metric="euclidean", cluster_selection_method=c["method"],
                              prediction_data=True),
        vectorizer_model=CountVectorizer(ngram_range=(1, int(c["ngram_max"])), stop_words="english",
        min_df=2),
        tfidf_model=ClassTfidfTransformer(reduce_frequent_words=c["reduce_frequent_words"],
        bm25_weighting=c["bm25"]),
        representation_model=representation_model,
        nr_topics=c["nr_topics"], top_n_words=10, verbose=False)

def topic_word_lists(model, aspect=None, topn=10):
    topics = model.get_topics() if aspect is None else model.topic_aspects_[aspect]
    res = {}
    for tid, ws in topics.items():
        if tid == -1:
            continue
        toks = []
        for w, _ in ws:
            for t in str(w).split(): # split n-grams into unigrams for scoring
                if t and t not in toks:
                    toks.append(t)
        if toks:
            res[tid] = toks[:topn]
    return res

def evaluate(model, aspect=None):
    tw = topic_word_lists(model, aspect)
    cv_per = {t: CV.score_topic(w, "c_v") for t, w in tw.items()}
    np_per = {t: NP.score_topic(w, "c_npmi") for t, w in tw.items()}

```

```

words = [w for ws in tw.values() for w in ws]
m = dict(n_topics=len(tw),
        c_v=np.nanmean(list(cv_per.values())) if tw else np.nan,
        c_npmi=np.nanmean(list(np_per.values())) if tw else np.nan,
        diversity=len(set(words)) / max(1, len(words)),
        outlier_pct=float(np.mean(np.asarray(model.topics_) == -1)),
        cv_per_topic=cv_per)
valid = m["n_topics"] >= MIN_TOPICS and m["outlier_pct"] <= MAX_OUTLIER_SHARE and not
np.isnan(m["c_v"])
m["score"] = m["c_v"] if valid else np.nan
return m

def run_config(overrides, stage, base=None, seed=SEED, keep_model=False, verbose=True):
    cfg = {**BASE_CFG, **(base or {}), **overrides}
    model = build_model(cfg, seed)
    t0 = time.time()
    try:
        model.fit(DOCS, embeddings=EMB[cfg["emb"]])
        m = evaluate(model)
    except Exception as e:
        print(f" [{stage}] fit failed: {e}")
        m = dict(n_topics=0, c_v=np.nan, c_npmi=np.nan, diversity=np.nan, outlier_pct=np.nan,
score=np.nan)
    rec = {"stage": stage, "seed": seed, "cfg": dict(cfg), **cfg,
        **{k: v for k, v in m.items() if k != "cv_per_topic"}, "seconds": time.time() - t0}
    LOG.append(rec)
    if verbose:
        flag = "" if not np.isnan(rec["score"]) else " (fails constraints)"
        print(f" [{stage}] c_v={rec['c_v']:.3f} npmi={rec['c_npmi']:.3f} topics=
{rec['n_topics']}>2} "
            f"outliers={rec['outlier_pct']:.0%}{flag}")
    return (model if keep_model else None), rec

def best_record(stages=None, exclude=("G Stability",)):
    recs = [r for r in LOG if (stages is None or r["stage"] in stages) and r["stage"] not in exclude]
    pool = [r for r in recs if not np.isnan(r["score"])] or [r for r in recs if not
np.isnan(r["c_v"])]
    if not pool:
        raise RuntimeError("No successful BERTopic fit in these stages.")
    return max(pool, key=lambda r: r["score"] if not np.isnan(r["score"]) else r["c_v"])

def stage_df(stage):
    return pd.DataFrame([r for r in LOG if r["stage"] == stage])

```

12. BERTopic Optimisation (target: c_v coherence)

Stages A-E are a **coordinate-descent** search: each stage tunes one group of settings and passes its winner on. Stage F runs a **joint search** over everything at once, comparing **Random search** with **Bayesian optimisation (Optuna TPE)**. Stage G checks that the winner is stable across random seeds.

Baseline — the original notebook's setup

all-mpnet-base-v2 on the cleaned text, BERTopic defaults.

In [18]:

```
_, baseline = run_config({"emb": "mpnet|cleaned"}, stage="0 Baseline")
```

```
[0 Baseline] c_v=0.410 npmi=-0.095 topics=12 outliers=31%
```

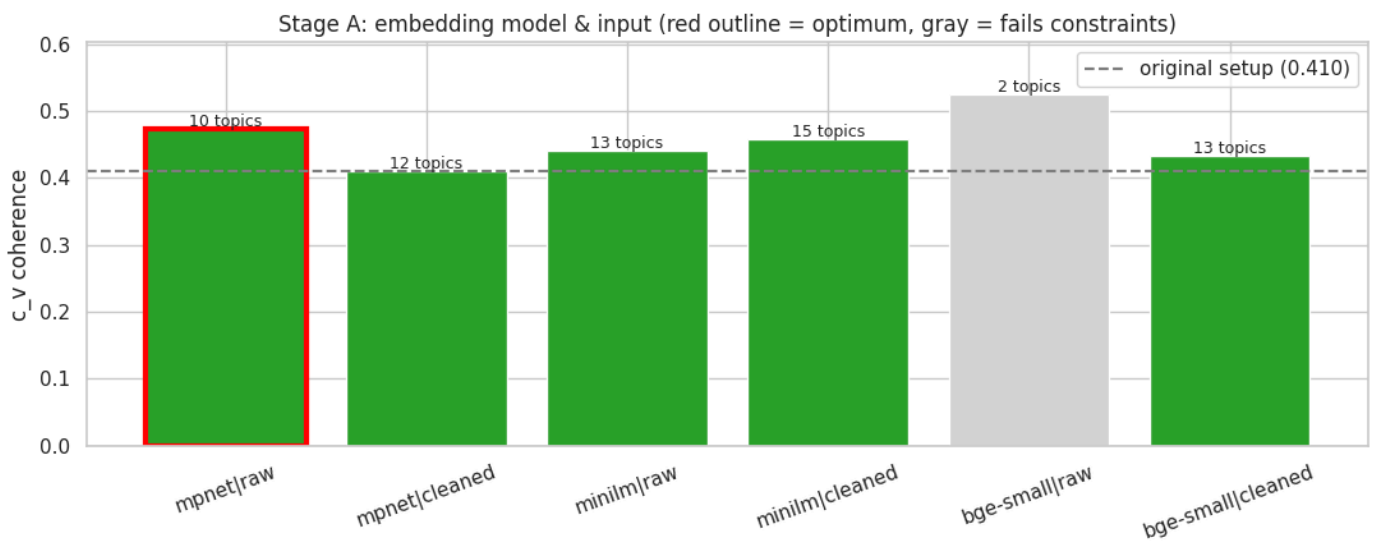
Stage A — embedding model & input text

In [19]:

```
for k in EMB:
    run_config({"emb": k}, stage="A Embedding")

a = stage_df("A Embedding")
best_a = best_record(["A Embedding"])
fig, ax = plt.subplots(figsize=(11, 4.5))
colors = ["tab:green" if not np.isnan(s) else "lightgray" for s in a["score"]]
bars = ax.bar(a["emb"], np.nan_to_num(a["c_v"]), color=colors)
for b, n_t, e in zip(bars, a["n_topics"], a["emb"]):
    ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.004, f"{n_t} topics", ha="center",
            fontsize=9)
    if e == best_a["emb"]:
        b.set_edgecolor("red"); b.set_linewidth(3)
ax.axhline(baseline["c_v"], ls="--", color="gray", label=f"original setup ({baseline['c_v']:.3f})")
ax.set_ylabel("c_v coherence"); ax.set_ylim(0, max(np.nan_to_num(a["c_v"])) * 1.15)
ax.set_title("Stage A: embedding model & input (red outline = optimum, gray = fails constraints)")
ax.legend(); plt.xticks(rotation=20); plt.tight_layout(); plt.show()
print("Best embedding:", best_a["emb"])
```

[A Embedding]	c_v=0.474	npmi=-0.019	topics=10	outliers=27%
[A Embedding]	c_v=0.410	npmi=-0.095	topics=12	outliers=31%
[A Embedding]	c_v=0.441	npmi=-0.064	topics=13	outliers=28%
[A Embedding]	c_v=0.458	npmi=-0.044	topics=15	outliers=30%
[A Embedding]	c_v=0.524	npmi=-0.010	topics= 2	outliers=2% (fails constraints)
[A Embedding]	c_v=0.434	npmi=-0.058	topics=13	outliers=29%



Best embedding: mpnet|raw

Stage B — HDBSCAN min_cluster_size grid

In [20]:

```
cur = best_a["cfg"]
for mcs in [5, 8, 10, 12, 15, 20, 25, 30, 40]:
    run_config({"min_cluster_size": mcs}, stage="B HDBSCAN min_cluster_size", base=cur)

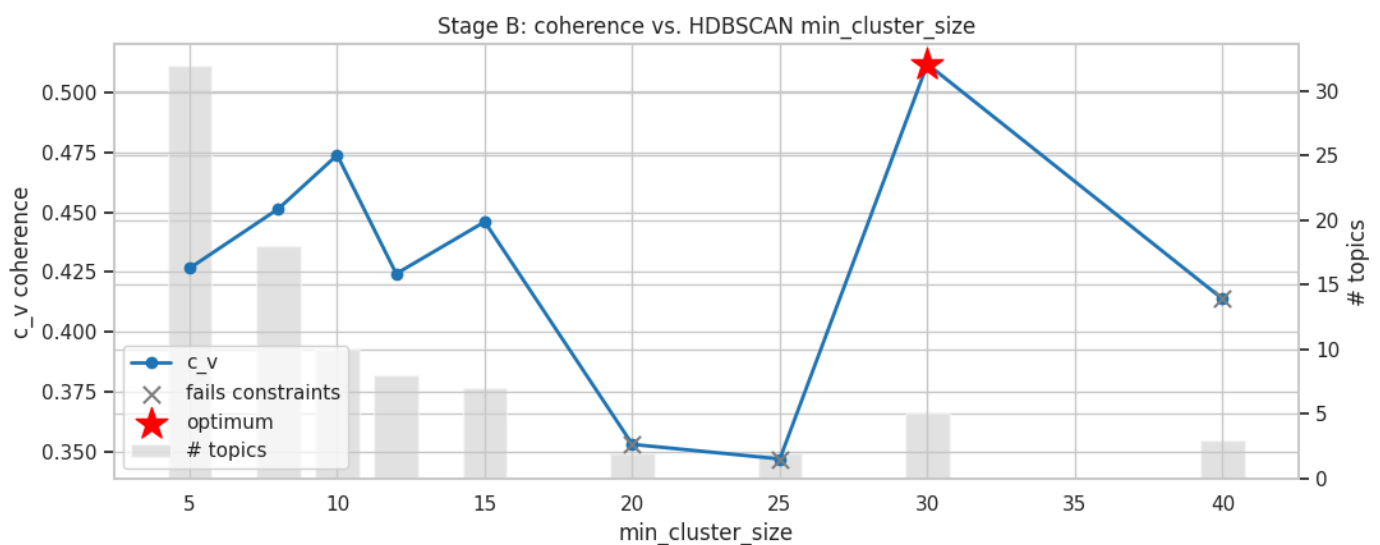
b = stage_df("B HDBSCAN min_cluster_size").sort_values("min_cluster_size")
best_b = best_record(["B HDBSCAN min_cluster_size"])
fig, ax1 = plt.subplots(figsize=(11, 4.5))
ax2 = ax1.twinx()
ax2.bar(b["min_cluster_size"], b["n_topics"], width=1.5, color="lightgray", alpha=0.6, label="# topics")
ax1.plot(b["min_cluster_size"], b["c_v"], "o-", color="tab:blue", lw=2, label="c_v")
invalid = b[b["score"].isna()]
```

```

ax1.scatter(invalid["min_cluster_size"], invalid["c_v"], marker="x", s=90, color="gray", zorder=5,
label="fails constraints")
ax1.scatter([best_b["min_cluster_size"]], [best_b["c_v"]], marker="*", s=350, color="red", zorder=6,
label="optimum")
ax1.set_zorder(ax2.get_zorder() + 1); ax1.patch.set_visible(False)
ax1.set_xlabel("min_cluster_size"); ax1.set_ylabel("c_v coherence"); ax2.set_ylabel("# topics")
h1, l1 = ax1.get_legend_handles_labels(); h2, l2 = ax2.get_legend_handles_labels()
ax1.legend(h1 + h2, l1 + l2, loc="best")
ax1.set_title("Stage B: coherence vs. HDBSCAN min_cluster_size")
plt.tight_layout(); plt.show()

```

[B HDBSCAN min_cluster_size]	c_v	npmi	topics	outliers	status
5	0.427	-0.087	32	23%	
10	0.451	-0.060	18	32%	
15	0.474	-0.019	10	27%	
20	0.424	-0.074	8	18%	
25	0.446	-0.053	7	20%	
30	0.353	-0.016	2	8%	(fails constraints)
35	0.347	-0.017	2	10%	(fails constraints)
40	0.512	-0.007	5	43%	
45	0.414	-0.034	3	64%	(fails constraints)



Stage C — UMAP `n_neighbors` × `n_components` grid

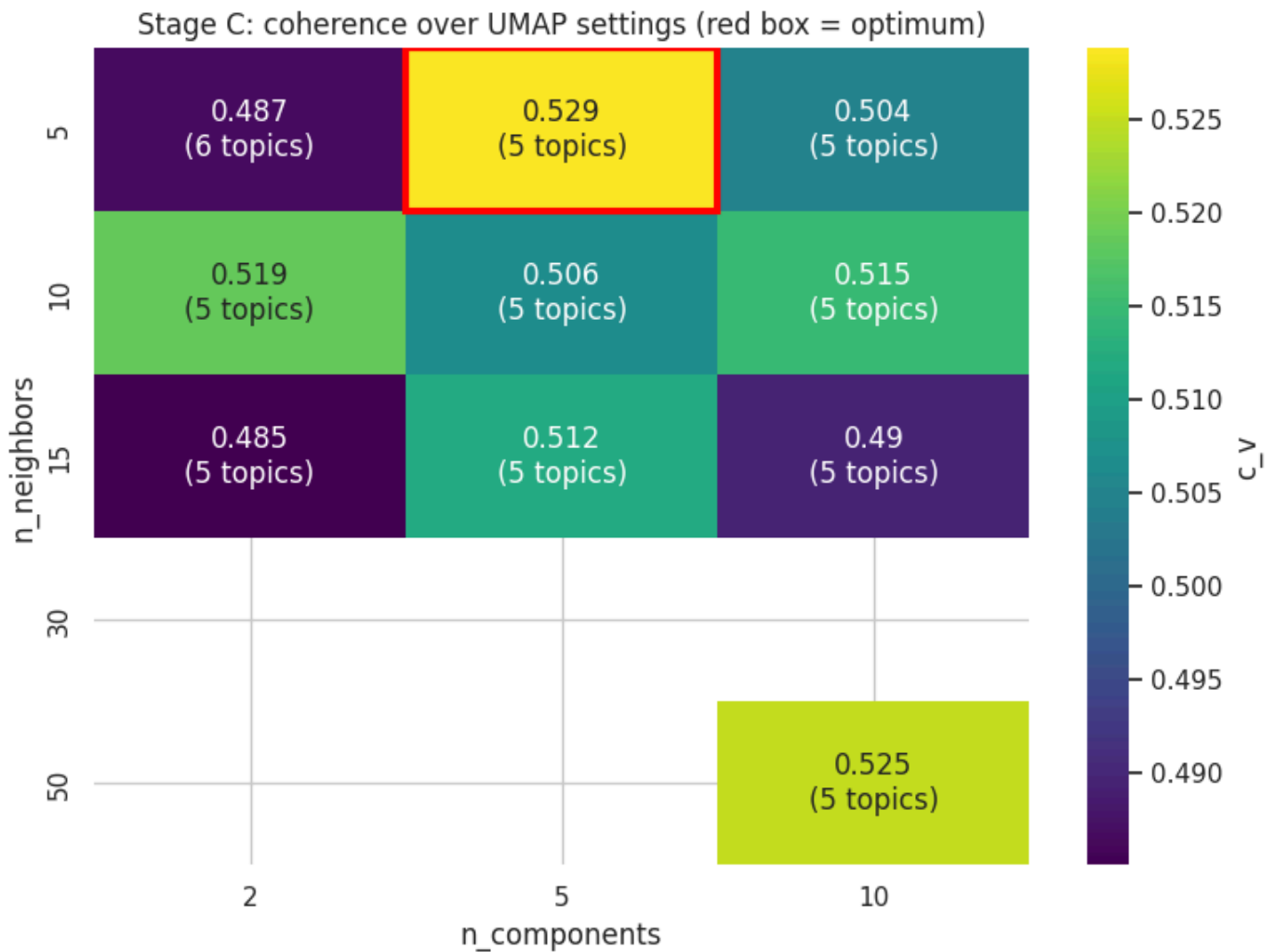
In [21]:

```

cur = best_b["cfg"]
for nn in [5, 10, 15, 30, 50]:
    for nc in [2, 5, 10]:
        run_config({"n_neighbors": nn, "n_components": nc}, stage="C UMAP grid", base=cur,
        verbose=False)

c = stage_df("C UMAP grid")
best_c = best_record(["C UMAP grid"])
piv = c.pivot_table(index="n_neighbors", columns="n_components", values="score", dropna=False)
piv_n = c.pivot_table(index="n_neighbors", columns="n_components", values="n_topics",
dropna=False).fillna(0)
annot = piv.round(3).astype(str) + "\n(" + piv_n.astype(int).astype(str) + " topics)"
annot = annot.where(piv.notna(), "fails")
plt.figure(figsize=(8, 6))
ax = sns.heatmap(piv, annot=annot, fmt="", cmap="viridis", cbar_kws={"label": "c_v"})
r_i = list(piv.index).index(best_c["n_neighbors"]); c_i =
list(piv.columns).index(best_c["n_components"])
ax.add_patch(plt.Rectangle((c_i, r_i), 1, 1, fill=False, edgecolor="red", lw=3))
ax.set_title("Stage C: coherence over UMAP settings (red box = optimum)")
plt.tight_layout(); plt.show()
print(f"Best UMAP: n_neighbors={best_c['n_neighbors']}, n_components={best_c['n_components']}, c_v=
{best_c['c_v']:.3f}")

```



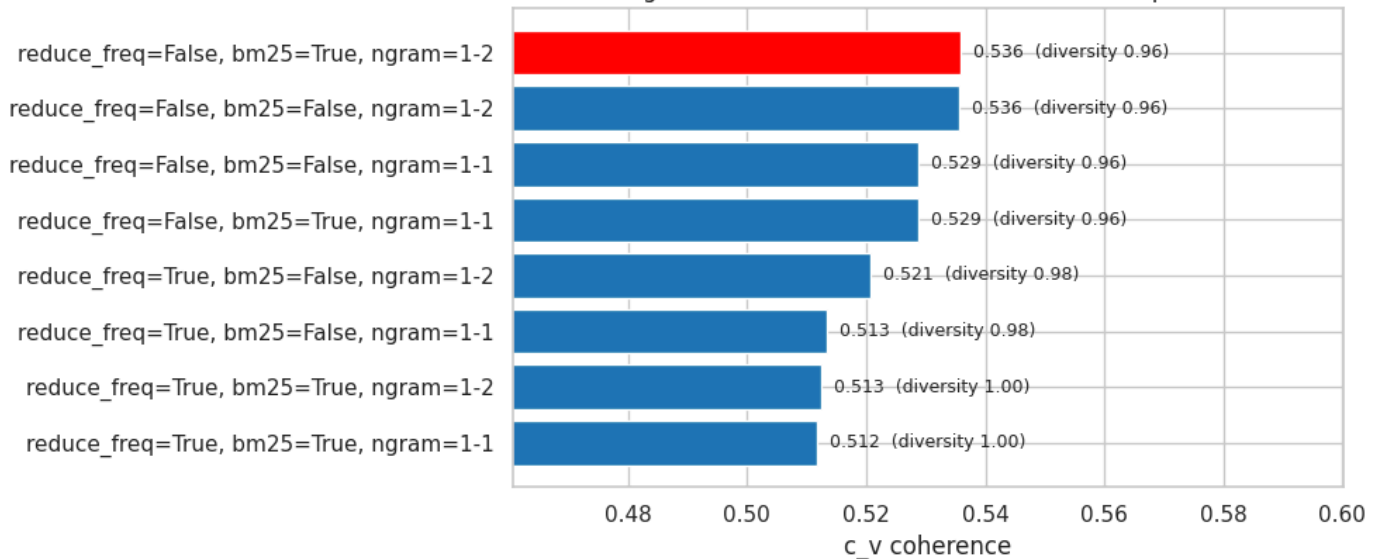
Stage D — topic-word extraction (c-TF-IDF variants & n-grams)

In [22]:

```
cur = best_c["cfg"]
for rfw in (False, True):
    for bm25 in (False, True):
        for ng in (1, 2):
            run_config({"reduce_frequent_words": rfw, "bm25": bm25, "ngram_max": ng},
                        stage="D c-TF-IDF variants", base=cur, verbose=False)

d = stage_df("D c-TF-IDF variants")
d["variant"] = d.apply(lambda r: f"reduce_freq={r.reduce_frequent_words}, bm25={r.bm25}, ngram=1-{r.ngram_max}", axis=1)
d = d.sort_values("c_v")
best_d = best_record(["D c-TF-IDF variants"])
fig, ax = plt.subplots(figsize=(10, 4.5))
cols = ["red" if (r.reduce_frequent_words == best_d["reduce_frequent_words"] and r.bm25 == best_d["bm25"]
                and r.ngram_max == best_d["ngram_max"]) else "tab:blue" for r in d.itertuples()]
ax.barh(d["variant"], d["c_v"], color=cols)
for y_, (v, dv) in enumerate(zip(d["c_v"], d["diversity"])):
    ax.text(v + 0.002, y_, f"{v:.3f} (diversity {dv:.2f})", va="center", fontsize=9)
ax.set_xlim(d["c_v"].min() * 0.9, d["c_v"].max() * 1.12)
ax.set_xlabel("c_v coherence"); ax.set_title("Stage D: c-TF-IDF / vectorizer variants (red = optimum)")
plt.tight_layout(); plt.show()
```

Stage D: c-TF-IDF / vectorizer variants (red = optimum)



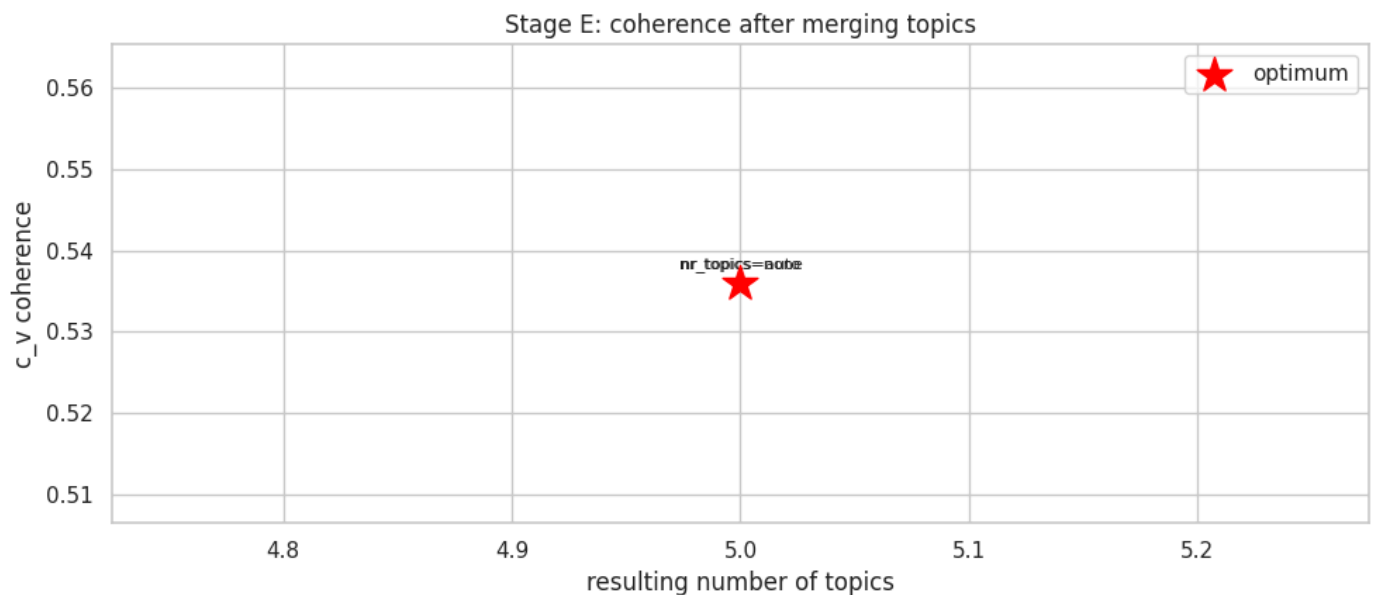
Stage E — topic reduction (nr_topics)

Merging similar topics can raise or lower coherence; this sweep finds out which.

In [23]:

```
cur = best_d["cfg"]
n_base = int(best_d["n_topics"])
targets = [None, "auto"]
if n_base - 1 > MIN_TOPICS:
    targets += sorted(set(np.linspace(MIN_TOPICS + 1, n_base, 6).astype(int).tolist()))
for k in targets:
    run_config({"nr_topics": k}, stage="E Topic reduction", base=cur, verbose=False)

e = stage_df("E Topic reduction")
best_e = best_record(["E Topic reduction"])
e["setting"] = e["nr_topics"].apply(lambda v: "none" if v is None or (isinstance(v, float) and
np.isnan(v)) else str(v))
e_s = e.sort_values("nr_topics")
fig, ax = plt.subplots(figsize=(10, 4.5))
ax.plot(e_s["nr_topics"], e_s["c_v"], "o-", lw=2)
for r in e_s.itertuples():
    ax.annotate(f"nr_topics={r.setting}", (r.nr_topics, r.c_v), textcoords="offset points", xytext=(0,
8), ha="center", fontsize=8)
ax.scatter([best_e["nr_topics"]], [best_e["c_v"]], marker="*", s=350, color="red", zorder=5,
label="optimum")
ax.set_xlabel("resulting number of topics"); ax.set_ylabel("c_v coherence")
ax.set_title("Stage E: coherence after merging topics"); ax.legend()
plt.tight_layout(); plt.show()
```



Stage F — joint search: Random search vs. Bayesian optimisation (Optuna TPE)

Both search all settings at once (embedding, UMAP, HDBSCAN incl. `min_samples` and leaf selection, c-TF-IDF options, n-grams) with the same budget of `N_TRIALS` fits.

In [24]:

```
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)

def make_objective(stage):
    def objective(trial):
        mcs = trial.suggest_int("min_cluster_size", 5, 40)
        cfg = dict(
            emb=trial.suggest_categorical("emb", list(EMB)),
            n_neighbors=trial.suggest_int("n_neighbors", 5, 60),
            n_components=trial.suggest_int("n_components", 2, 15),
            min_cluster_size=mcs,
            min_samples=trial.suggest_int("min_samples", 1, mcs),
            method=trial.suggest_categorical("method", ["eom", "leaf"]),
            ngram_max=trial.suggest_int("ngram_max", 1, 2),
            reduce_frequent_words=trial.suggest_categorical("reduce_frequent_words", [False, True]),
            bm25=trial.suggest_categorical("bm25", [False, True]),
            nr_topics=None
        ), rec = run_config(cfg, stage, verbose=False)
        if (trial.number + 1) % 10 == 0:
            print(f" {stage}: trial {trial.number + 1}/{N_TRIALS}")
        return 0.0 if np.isnan(rec["score"]) else rec["score"]
    return objective

t0 = time.time()
study_rand = optuna.create_study(direction="maximize",
sampler=optuna.samplers.RandomSampler(seed=SEED))
study_rand.optimize(make_objective("F Random search"), n_trials=N_TRIALS)
study_tpe = optuna.create_study(direction="maximize",
                                sampler=optuna.samplers.TPESampler(seed=SEED, n_startup_trials=8,
multivariate=True))
study_tpe.optimize(make_objective("F Bayesian (TPE)"), n_trials=N_TRIALS)
print(f"Joint search done in {time.time() - t0:.0f}s | best random={study_rand.best_value:.3f} | best
TPE={study_tpe.best_value:.3f}")
```

```

F Random search: trial 10/40
F Random search: trial 20/40
F Random search: trial 30/40
F Random search: trial 40/40
F Bayesian (TPE): trial 10/40
F Bayesian (TPE): trial 20/40
F Bayesian (TPE): trial 30/40
F Bayesian (TPE): trial 40/40
Joint search done in 251s | best random=0.528 | best TPE=0.520

```

In [25]:

```

def best_so_far(vals):
    out, cur = [], -np.inf
    for v in vals:
        if not np.isnan(v):
            cur = max(cur, v)
        out.append(cur if np.isfinite(cur) else np.nan)
    return out

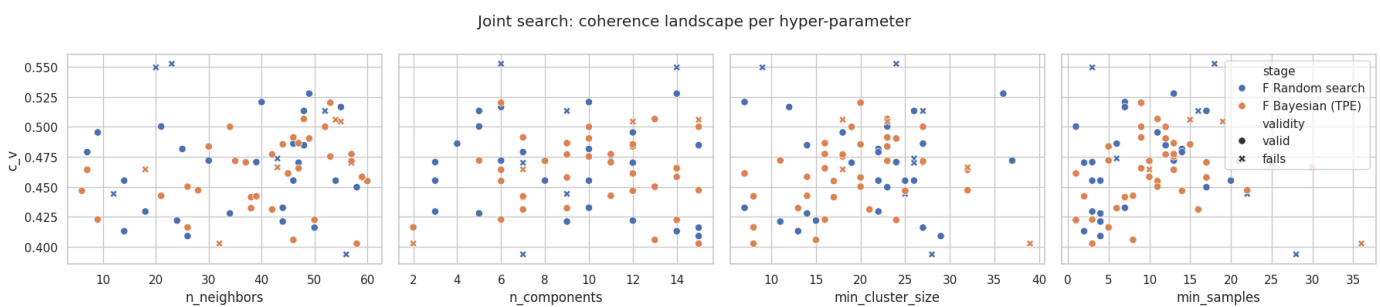
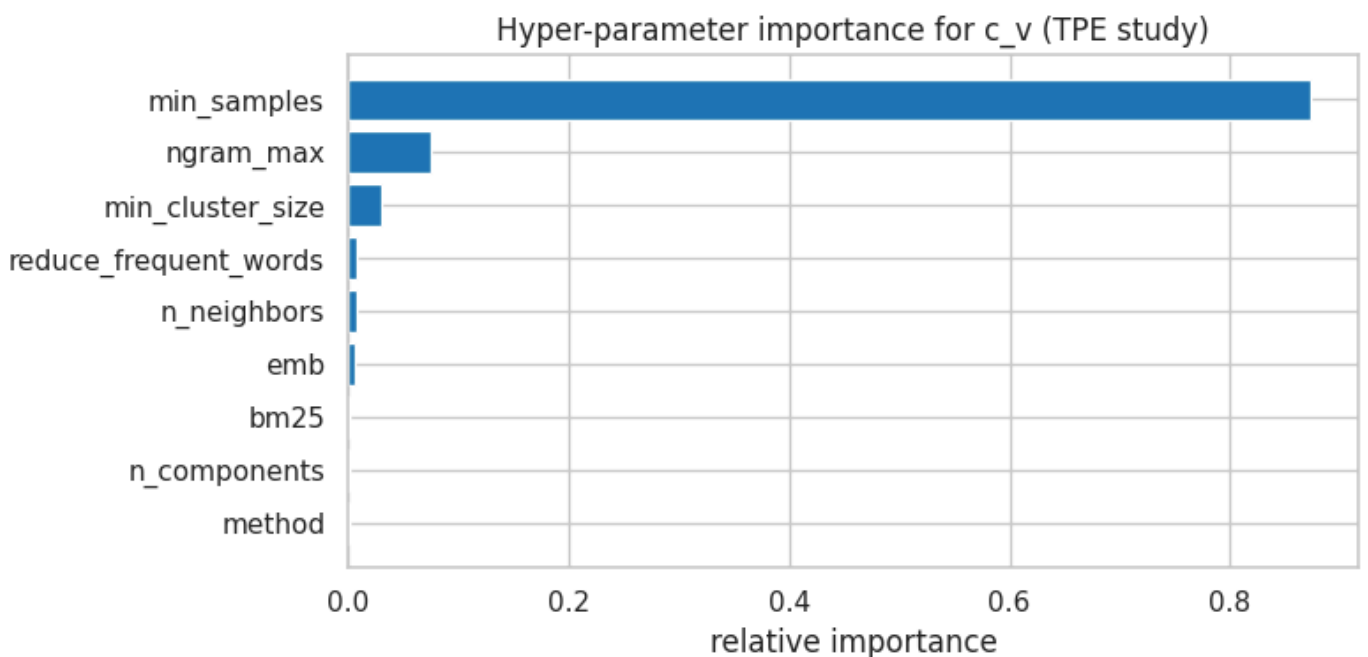
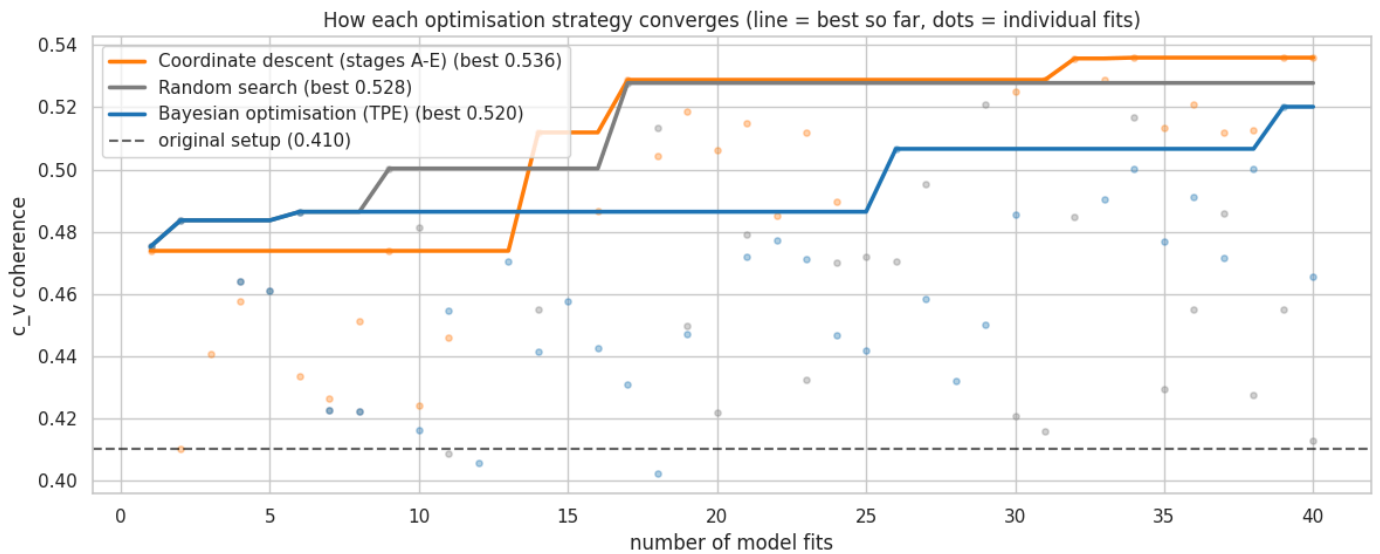
curves = {
    "Coordinate descent (stages A-E)": [r["score"] for r in LOG if r["stage"][0] in "ABCDE"],
    "Random search": [r["score"] for r in LOG if r["stage"] == "F Random search"],
    "Bayesian optimisation (TPE)": [r["score"] for r in LOG if r["stage"] == "F Bayesian (TPE)"],
}

fig, ax = plt.subplots(figsize=(12, 5))
for (name, vals), col in zip(curves.items(), ["tab:orange", "tab:gray", "tab:blue"]):
    x = np.arange(1, len(vals) + 1)
    ax.scatter(x, vals, s=14, alpha=0.35, color=col)
    ax.plot(x, best_so_far(vals), lw=2.5, color=col, label=f"{name} (best {np.nanmax(vals):.3f})")
ax.axhline(baseline["c_v"], ls="--", color="black", alpha=0.6, label=f"original setup ({baseline['c_v']:.3f})")
ax.set_xlabel("number of model fits"); ax.set_ylabel("c_v coherence")
ax.set_title("How each optimisation strategy converges (line = best so far, dots = individual fits)")
ax.legend(); plt.tight_layout(); plt.show()

# Which hyper-parameters mattered most (fANOVA on the TPE study)
try:
    imp = optuna.importance.get_param_importances(study_tpe)
    plt.figure(figsize=(8, 4))
    plt.barh(list(imp.keys())[:-1], list(imp.values())[:-1], color="tab:blue")
    plt.xlabel("relative importance"); plt.title("Hyper-parameter importance for c_v (TPE study)")
    plt.tight_layout(); plt.show()
except Exception as ex:
    print("Importance plot skipped:", ex)

# Coherence landscape of all joint-search trials
f = pd.concat([stage_df("F Random search"), stage_df("F Bayesian (TPE)")]).reset_index(drop=True)
f["validity"] = f["score"].isna().map({True: "fails", False: "valid"})
fig, axes = plt.subplots(1, 4, figsize=(18, 4), sharey=True)
for ax, p in zip(axes, ["n_neighbors", "n_components", "min_cluster_size", "min_samples"]):
    sns.scatterplot(data=f, x=p, y="c_v", hue="stage", style="validity", ax=ax, s=45)
    leg = ax.get_legend()
    if leg is not None and ax is not axes[-1]:
        leg.remove()
axes[0].set_ylabel("c_v")
fig.suptitle("Joint search: coherence landscape per hyper-parameter"); plt.tight_layout(); plt.show()

```



Stage G — seed stability of the overall winner

In [26]:

```
overall = best_record()
print("Overall best stage:", overall["stage"])
print("Config:", {k: overall["cfg"][k] for k in BASE_CFG})
print(f"c_v={overall['c_v']:.3f}  c_npmi={overall['c_npmi']:.3f}  topics={overall['n_topics']}")
outliers={overall['outlier_pct']:.0%})

stab_seeds = [0, 1, 2] if FAST_MODE else [0, 1, 2, 3, 4]
```

```

for s in stab_seeds:
    run_config({}, stage="G Stability", base=overall["cfg"], seed=s, verbose=False)
g = stage_df("G Stability")

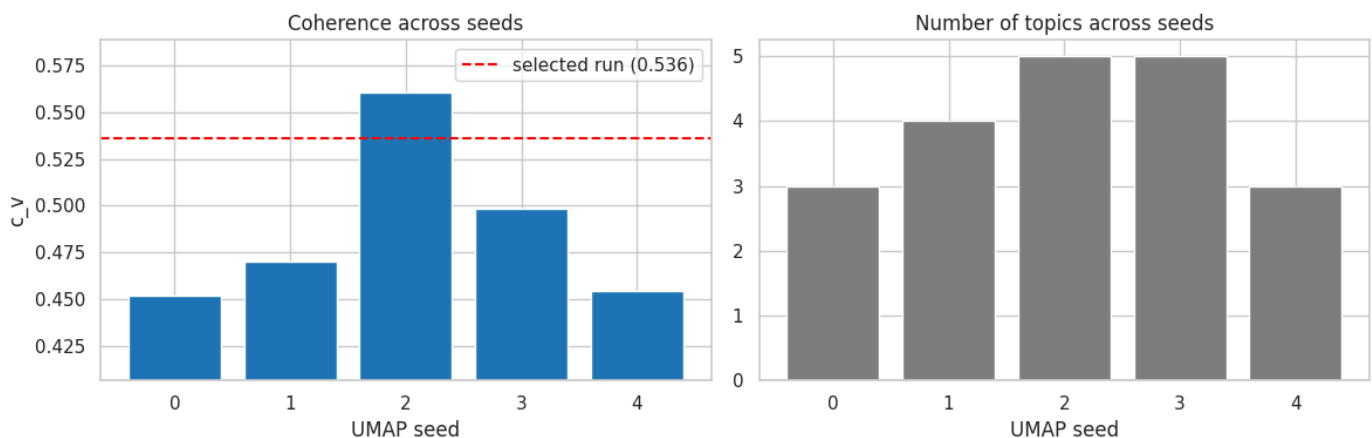
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].bar(g["seed"].astype(str), g["c_v"], color="tab:blue")
axes[0].axhline(overall["c_v"], color="red", ls="--", label=f"selected run ({overall['c_v']:.3f})")
axes[0].set_ylim(min(g["c_v"].min(), overall["c_v"]) * 0.9, max(g["c_v"].max(), overall["c_v"]) * 1.05)
axes[0].set_xlabel("UMAP seed"); axes[0].set_ylabel("c_v"); axes[0].legend();
axes[0].set_title("Coherence across seeds")
axes[1].bar(g["seed"].astype(str), g["n_topics"], color="tab:gray")
axes[1].set_xlabel("UMAP seed"); axes[1].set_title("Number of topics across seeds")
plt.tight_layout(); plt.show()
print(f"Across seeds: c_v mean {g['c_v'].mean():.3f} ± {g['c_v'].std():.3f}")

```

Overall best stage: D c-TF-IDF variants

Config: {'emb': 'mpnet|raw', 'n_neighbors': 5, 'n_components': 5, 'min_cluster_size': 30, 'min_samples': None, 'method': 'eom', 'ngram_max': 2, 'reduce_frequent_words': False, 'bm25': True, 'nr_topics': None}

c_v=0.536 c_npmi=-0.033 topics=5 outliers=39%



Across seeds: c_v mean 0.487 ± 0.045

Optimisation summary

In [27]:

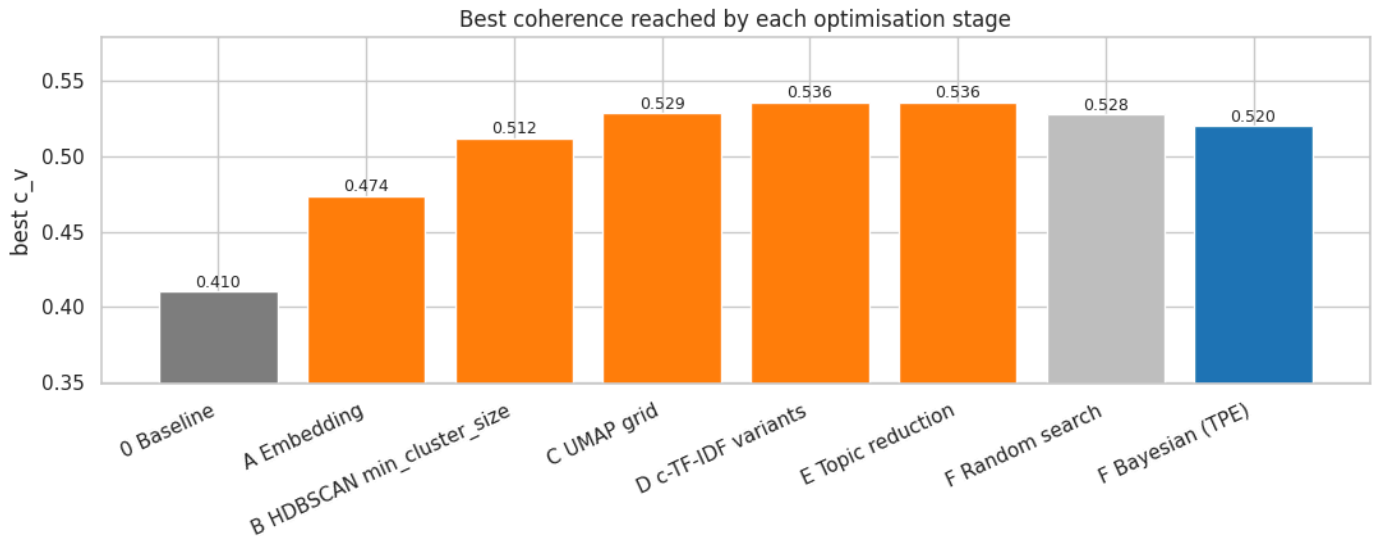
```

stages = ["0 Baseline", "A Embedding", "B HDBSCAN min_cluster_size", "C UMAP grid", "D c-TF-IDF variants",
          "E Topic reduction", "F Random search", "F Bayesian (TPE)"]
summ = []
for s in stages:
    try:
        r = best_record([s])
        summ.append({"stage": s, "best c_v": r["c_v"], "c_npmi": r["c_npmi"], "topics": r["n_topics"],
                    "outliers": r["outlier_pct"], "diversity": r["diversity"]})
    except RuntimeError:
        pass
summ = pd.DataFrame(summ)
fig, ax = plt.subplots(figsize=(11, 4.5))
stage_color = lambda s: "gray" if s.startswith("0") else ("tab:blue" if "TPE" in s else ("silver" if "Random" in s else "tab:orange"))
bars = ax.bar(summ["stage"], summ["best c_v"], color=[stage_color(s) for s in summ["stage"]])
for b, v in zip(bars, summ["best c_v"]):
    ax.text(b.get_x() + b.get_width() / 2, v + 0.003, f"{v:.3f}", ha="center", fontsize=9)
ax.set_ylim(summ["best c_v"].min() * 0.85, summ["best c_v"].max() * 1.08)
ax.set_ylabel("best c_v"); ax.set_title("Best coherence reached by each optimisation stage")
plt.xticks(rotation=25, ha="right"); plt.tight_layout(); plt.show()
display(summ.round(3))

log_df = pd.DataFrame([k: v for k, v in r.items() if k != "cfg"] for r in LOG])

```

```
log_df.to_csv("/content/bertopic_optimisation_log.csv", index=False)
display(log_df[log_df["stage"] != "G Stability"].sort_values("score", ascending=False)
        .head(10)[["stage", "emb", "n_neighbors", "n_components", "min_cluster_size", "min_samples",
"method",
                    "ngram_max", "reduce_frequent_words", "bm25", "nr_topics", "c_v", "c_npmi",
                    "n_topics", "outlier_pct"]]
        .round(3))
```



	stage	best c_v	c_npmi	topics	outliers	diversity
0	0 Baseline	0.410	-0.095	12	0.311	0.925
1	A Embedding	0.474	-0.019	10	0.266	0.920
2	B HDBSCAN min_cluster_size	0.512	-0.007	5	0.430	0.960
3	C UMAP grid	0.529	-0.040	5	0.389	0.960
4	D c-TF-IDF variants	0.536	-0.033	5	0.389	0.957
5	E Topic reduction	0.536	-0.033	5	0.389	0.957
6	F Random search	0.528	-0.059	5	0.408	0.979
7	F Bayesian (TPE)	0.520	-0.008	8	0.494	0.958

	stage	emb	n_neighbors	n_components	n_min_cluster_size	min_samples	method	ngram_max	reduce_frequent_words	bm25	nr_topics	c_v	c_npmi	n_topics	outlier_pct
34	D c-TF-IDF variants	mpnet raw	5	5	30	NaN	eom	2	False	True	None	0.536	-0.033	5	0.389
40	E Topic reduction	mpnet raw	5	5	30	NaN	eom	2	False	True	auto	0.536	-0.033	5	0.389
39	E Topic reduction	mpnet raw	5	5	30	NaN	eom	2	False	True	None	0.536	-0.033	5	0.389
32	D c-TF-IDF variants	mpnet raw	5	5	30	NaN	eom	2	False	False	None	0.536	-0.033	5	0.389
17	C UMAP grid	mpnet raw	5	5	30	NaN	eom	1	False	False	None	0.529	-0.040	5	0.389
33	D c-TF-IDF variants	mpnet raw	5	5	30	NaN	eom	1	False	True	None	0.529	-0.040	5	0.389
31	D c-TF-IDF variants	mpnet raw	5	5	30	NaN	eom	1	False	False	None	0.529	-0.040	5	0.389
57	F Random search	mpnet raw	49	14	36	13.0	eom	2	True	False	None	0.528	-0.059	5	0.408
30	C UMAP grid	mpnet raw	50	10	30	NaN	eom	1	False	False	None	0.525	-0.014	5	0.440
36	D c-TF-IDF variants	mpnet raw	5	5	30	NaN	eom	2	True	False	None	0.521	-0.042	5	0.389

13. Final Model & Readable Topics (no LLM API)

1. Refit the winning configuration and add two extra keyword representations (**KeyBERTInspired**, **MaximalMarginalRelevance**); keep whichever is most coherent for display.
2. Score every Biden sentence with Twitter-RoBERTa to get each topic's sentiment mix.
3. Give each topic a short title with **google/flan-t5-large**, a free ~780M-parameter model that runs locally in the notebook (no API key, no LLM service). Its input is the topic's keywords plus its most representative sentences.

In [28]:

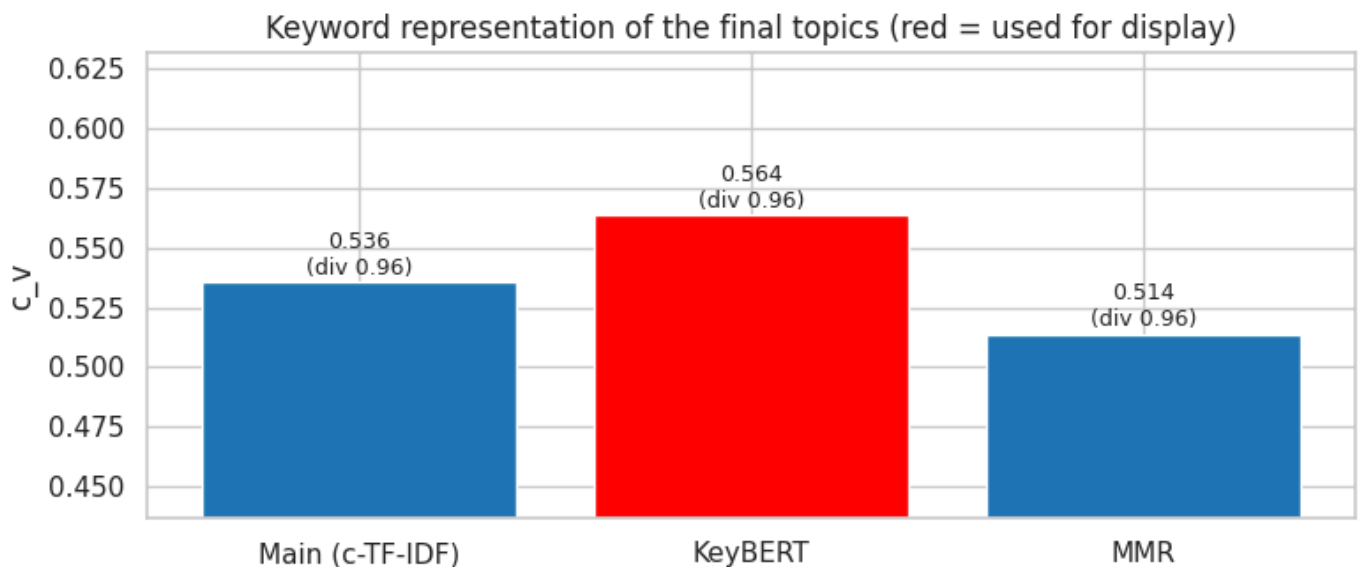
```
from bertopic.representation import KeyBERTInspired, MaximalMarginalRelevance

FINAL_CFG = dict(overall["cfg"])
final_model = build_model(
    FINAL_CFG, seed=overall["seed"],
    representation_model={
        "KeyBERT": KeyBERTInspired(), "MMR":
        MaximalMarginalRelevance(diversity=0.3)},
    embedding_model=ST[FINAL_CFG["emb"].split("|")[0]]
)
final_model.fit(DOCS, embeddings=EMB[FINAL_CFG["emb"]])
final_metrics = evaluate(final_model)
print(f"Final model: {final_metrics['n_topics']} topics | c_v={final_metrics['c_v']:.3f} "
      f"| c_npmi={final_metrics['c_npmi']:.3f} | outliers={final_metrics['outlier_pct']:.0%}")

rep_rows = []
for asp in ["Main (c-TF-IDF)", "KeyBERT", "MMR"]:
    m = evaluate(final_model, None if asp.startswith("Main") else asp)
    rep_rows.append(
        {"representation": asp, "c_v": m["c_v"], "c_npmi": m["c_npmi"], "diversity":
        m["diversity"]}
    )
rep_df = pd.DataFrame(rep_rows)
BEST_ASPECT = rep_df.loc[rep_df["c_v"].idxmax(), "representation"]

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.bar(rep_df["representation"], rep_df["c_v"],
       color=["red" if r == BEST_ASPECT else "tab:blue" for r in rep_df["representation"]])
for i, (v, dv) in enumerate(zip(rep_df["c_v"], rep_df["diversity"])):
    ax.text(i, v + 0.003, f"{v:.3f}\n(div {dv:.2f})", ha="center", fontsize=9)
ax.set_ylim(rep_df["c_v"].min() * 0.85, rep_df["c_v"].max() * 1.12)
ax.set_ylabel("c_v"); ax.set_title("Keyword representation of the final topics (red = used for display)")
plt.tight_layout(); plt.show()
```

Final model: 5 topics | c_v=0.536 | c_npmi=-0.033 | outliers=39%



In [29]:

```
info = final_model.get_document_info(DOCS)
info["Sentence"] = DOCS_ORIG
info["Date"] = DOCS_DATES
info["Sentiment"], _ = roberta_sentiment(DOCS_ORIG)

KW = topic_word_lists(final_model, None if BEST_ASPECT.startswith("Main") else BEST_ASPECT, topn=10)
TOPIC_IDS = sorted(KW)
CV_TOPIC = {t: CV.score_topic(w, "c_v") for t, w in KW.items()}

# a few distinct, not-too-long sentences per topic
```

```
def examples_for(t, k=4):
    sub = info[info["Topic"] == t]
    if "Probability" in sub.columns:
        sub = sub.sort_values("Probability", ascending=False)
    seen, out = set(), []
    for s in sub["Sentence"]:
        key = s.lower()[:60]
        if key in seen:
            continue
        seen.add(key)
        if 6 <= len(s.split()) <= 60:
            out.append(s)
        if len(out) == k:
            break
    return out or list(sub["Sentence"].head(k))
```

In [30]:

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

lab_tok = AutoTokenizer.from_pretrained(LABEL_MODEL)
lab_model = AutoModelForSeq2SeqLM.from_pretrained(LABEL_MODEL).to(TORCH_DEVICE).eval()

def shorten(s, n=45):
    w = s.split()
    return " ".join(w[:n]) + (" ..." if len(w) > n else "")

def label_topic(keywords, examples):
    prompt = ("The sentences below come from political rally speeches and share one common theme.\n"
              f"Keywords: {' '.join(keywords[:8])}\n"
              "Sentences:\n" + "\n".join(f"- {shorten(s)}" for s in examples[:4])) +
            "\n\nWrite a short, neutral title of 2 to 6 words that describes the common theme."
    inputs = lab_tok(prompt, return_tensors="pt", truncation=True, max_length=512).to(TORCH_DEVICE)
    with torch.no_grad():
        out = lab_model.generate(**inputs, max_new_tokens=16, num_beams=4, no_repeat_ngram_size=2,
                                early_stopping=True)
        lab = lab_tok.decode(out[0], skip_special_tokens=True).strip().strip('."\'')
        # fall back to keywords if the model rambles
        if not lab or len(lab.split()) > 9:
            lab = " / ".join(keywords[:3])
        return lab[:1].upper() + lab[1:]

LABELS, used = {}, set()
for t in TOPIC_IDS:
    lab = label_topic(KW[t], examples_for(t))
    if lab.lower() in used:
        lab = f"{lab} ({KW[t][0]})"
    used.add(lab.lower())
    LABELS[t] = lab
    print(f"Topic {t:>2}: {lab} <- {' '.join(KW[t][:6])}")

final_model.set_topic_labels({**{t: f"{t}. {LABELS[t]}" for t in TOPIC_IDS}, -1: "Outliers"})
```

[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the checkpoints with different values, so we will NOT tie them. You should update the config with `tie\_word\_embeddings=False` to silence this warning.

```
Topic 0: Mobs, rioters, left-wing extremists <- violent, left, wing, radical, mob, far
Topic 1: Rally Speeches: Tax Cuts, Opposed <- tax, cuts, vote, voted, raise, taxes
Topic 2: Amnesty / destroy / healthcare <- amnesty, destroy, healthcare, mass, health, care
Topic 3: Biden's NAFTA and China deals wiped out one-third <- surrendered, jobs, economy, factories, industry, wants
Topic 4: Biden's anti-law enforcement rhetoric <- anti, police, departments, criminal, dismantle, law
```

In [31]:

```
rows = []
for t in TOPIC_IDS:
```

```

sub = info[info["Topic"] == t]
shares = sub["Sentiment"].value_counts(normalize=True)
ex = examples_for(t, 1)
rows.append({"Topic": t, "Label": LABELS[t], "Size": len(sub), "c_v": CV_TOPIC[t],
            "Negative": shares.get("negative", 0.0), "Neutral": shares.get("neutral", 0.0),
            "Positive": shares.get("positive", 0.0), "Keywords": ", ".join(KW[t][:8]),
            "Example": ex[0] if ex else ""})
topic_summary = pd.DataFrame(rows).sort_values("Size", ascending=False).reset_index(drop=True)
topic_summary.to_csv("/content/topic_summary.csv", index=False)
display(topic_summary.drop(columns=["Example"]).round(3))

```

	Topic	Label	Size	c_v	Negative	Neutral	Positive	Keywords
0	0	Mobs, rioters, left-wing extremists	163	0.704	0.638	0.337	0.025	violent, left, wing, radical, mob, far, rioter...
1	1	Rally Speeches: Tax Cuts, Opposed	136	0.558	0.515	0.390	0.096	tax, cuts, vote, voted, raise, taxes, election...
2	2	Amnesty / destroy / healthcare	53	0.552	0.774	0.226	0.000	amnesty, destroy, healthcare, mass, health, ca...
3	3	Biden's NAFTA and China deals wiped out one-third	50	0.432	0.880	0.100	0.020	surrendered, jobs, economy, factories, industr...
4	4	Biden's anti-law enforcement rhetoric	34	0.575	0.676	0.235	0.088	anti, police, departments, criminal, dismantle...

13.1 Topic cards (static, readable)

In [32]:

```

from matplotlib.patches import FancyBboxPatch
from matplotlib.colors import to_rgba

cmap = plt.cm.RdYlGn_r
ncols = 2
nrows = math.ceil(len(topic_summary) / ncols)
fig, axes = plt.subplots(nrows, ncols, figsize=(17, 3.2 * nrows))
axes = np.atleast_1d(axes).ravel()
for ax in axes:
    ax.axis("off")
for ax, r in zip(axes, topic_summary.itertuples()):
    ax.add_patch(FancyBboxPatch((0.01, 0.03), 0.98, 0.94,
                                transform=ax.transAxes, facecolor=to_rgba(cmap(r.Negative), 0.16),
                                edgecolor=cmap(r.Negative), lw=2))
    ax.text(0.04, 0.90, textwrap.shorten(f"Topic {r.Topic}: {r.Label}", 70), transform=ax.transAxes,
            fontsize=13.5, weight="bold", va="top")
    ax.text(0.04, 0.72, f"{r.Size} sentences | coherence {r.c_v:.2f} | "
                        f"{r.Negative:.0%} negative, {r.Neutral:.0%} neutral, {r.Positive:.0%}
positive",
            transform=ax.transAxes, fontsize=9.5, color="dimgray", va="top")
    ax.text(0.04, 0.56, textwrap.fill("Keywords: " + r.Keywords, 95), transform=ax.transAxes,
            fontsize=10.5, va="top")
    ax.text(0.04, 0.32, textwrap.fill('"' + textwrap.shorten(r.Example, 200) + '"', 110),
            transform=ax.transAxes, fontsize=9.5, style="italic", va="top", color="#333333")
fig.suptitle("What Trump says about Joe Biden: topics from the most coherent BERTopic model\n"
            "(card colour = share of negative sentences, green -> red)", fontsize=15, y=1.0)
plt.tight_layout(); plt.show()

```

What Trump says about Joe Biden: topics from the most coherent BERTopic model
(card colour = share of negative sentences, green -> red)

Topic 0: Mobs, rioters, left-wing extremists

163 sentences | coherence c_v 0.70 | 64% negative, 34% neutral, 2% positive

Keywords: violent, left, wing, radical, mob, far, rioters, party

"Biden has made a corrupt bargain exchange for his party's nomination, he's handed control to the Socialists, the Marxists, and the left-wing extremists and you know that and he's got no strength [...]"

Topic 1: Rally Speeches: Tax Cuts, Opposed

136 sentences | coherence c_v 0.56 | 51% negative, 39% neutral, 10% positive

Keywords: tax, cuts, vote, voted, raise, taxes, election, opposed

"You know our competitor, sleepy Joe, he had a rally today and practically nobody showed up."

Topic 2: Amnesty / destroy / healthcare

53 sentences | coherence c_v 0.55 | 77% negative, 23% neutral, 0% positive

Keywords: amnesty, destroy, healthcare, mass, health, care, plan, economy

"Sleepy Joe wants to quadruple your taxes, you know, wants to quadruple."

Topic 3: Biden's NAFTA and China deals wiped out one-third

50 sentences | coherence c_v 0.43 | 88% negative, 10% neutral, 2% positive

Keywords: surrendered, jobs, economy, factories, industry, wants, shipping, manufacturing

"In 1996, Joe Biden voted to obliterate Puerto Rico's thriving pharmaceutical industry."

Topic 4: Biden's anti-law enforcement rhetoric

34 sentences | coherence c_v 0.57 | 68% negative, 24% neutral, 9% positive

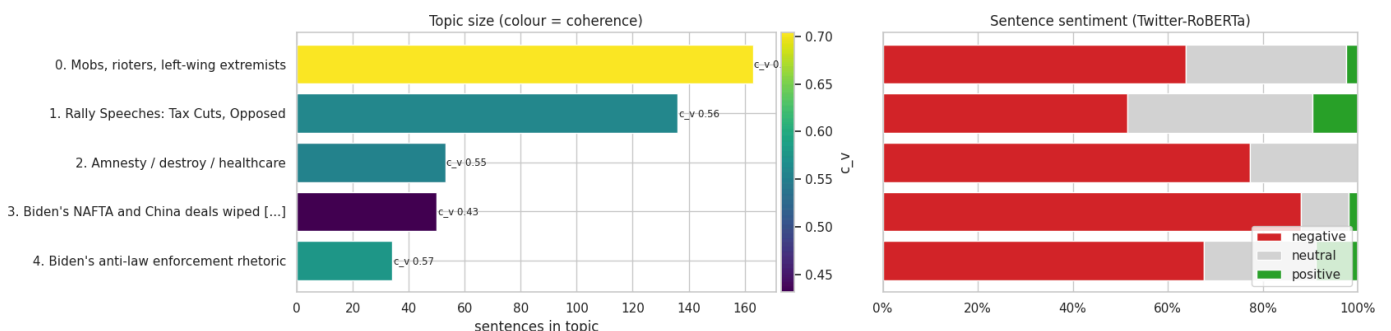
Keywords: anti, police, departments, criminal, dismantle, law, enforcement, criminals

"At our debate, Biden could not name a single law enforcement organization that supports him, not one."

13.2 Topic overview: size, coherence and sentiment

In [33]:

```
ts = topic_summary.sort_values("Size")
ylabels = [textwrap.shorten(f"{t}. {l}", 45) for t, l in zip(ts["Topic"], ts["Label"])]
fig, axes = plt.subplots(1, 2, figsize=(17, 0.45 * len(ts) + 2), sharey=True, gridspec_kw=
{"width_ratios": [1.2, 1]})
norm = plt.Normalize(ts["c_v"].min(), ts["c_v"].max())
axes[0].barh(ylabels, ts["Size"], color=plt.cm.viridis(norm(ts["c_v"])))
for y_, (s, v) in enumerate(zip(ts["Size"], ts["c_v"])):
    axes[0].text(s + 0.5, y_, f"c_v {v:.2f}", va="center", fontsize=8.5)
axes[0].set_xlabel("sentences in topic"); axes[0].set_title("Topic size (colour = coherence)")
sm = plt.cm.ScalarMappable(cmap="viridis", norm=norm); fig.colorbar(sm, ax=axes[0], label="c_v",
pad=0.01)
left = np.zeros(len(ts))
for col, color in (("Negative", "tab:red"), ("Neutral", "lightgray"), ("Positive", "tab:green")):
    axes[1].barh(ylabels, ts[col], left=left, color=color, label=col.lower()); left += ts[col].values
axes[1].set_xlim(0, 1); axes[1].xaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0%}"))
axes[1].set_title("Sentence sentiment (Twitter-RoBERTa)"); axes[1].legend(loc="lower right")
plt.tight_layout(); plt.show()
```



13.3 Interactive BERTopic views (with the generated labels)

In [34]:

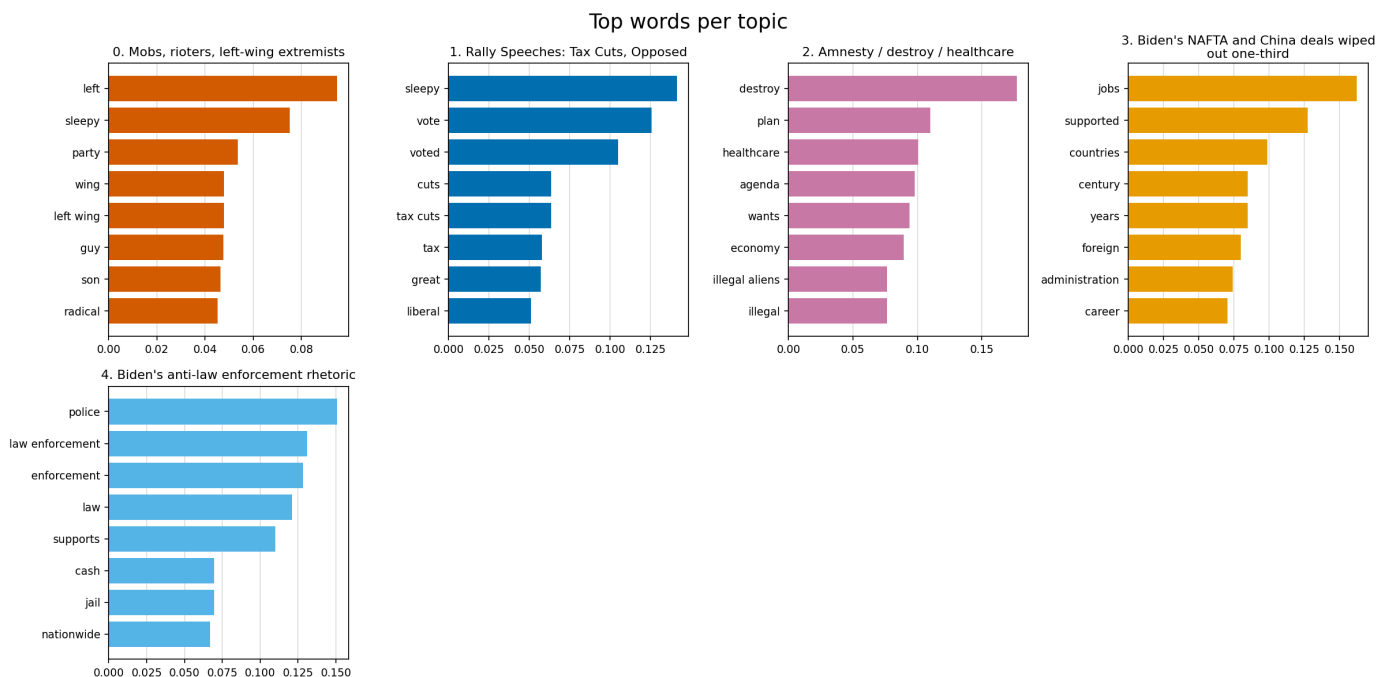
```
def safe_show(fn, name):
    try:
        fn().show()
    except Exception as ex:
        print(f"{name} skipped: {ex}")

safe_show(lambda: final_model.visualize_barchart(top_n_topics=min(12, len(TOPIC_IDS)), n_words=8,
        custom_labels=True, title="Top words per topic"),
        "Bar chart")

# 2d projection just for the document map
reduced_2d = UMAP(n_neighbors=15, n_components=2, min_dist=0.0, metric="cosine",
        random_state=SEED).fit_transform(EMB[FINAL_CFG["emb"]])
safe_show(lambda: final_model.visualize_documents(DOCS_ORIG, reduced_embeddings=reduced_2d,
        custom_labels=True,
        title="Biden sentences by topic (hover to read)",
        "Document map"))

safe_show(lambda: final_model.visualize_hierarchy(custom_labels=True, title="Topic hierarchy"),
        "Hierarchy")

if all(pd.notna(d) for d in DOCS_DATES):
    def _tot():
        tot = final_model.topics_over_time(DOCS, DOCS_DATES, nr_bins=12)
        return final_model.visualize_topics_over_time(tot, top_n_topics=min(8, len(TOPIC_IDS)),
        custom_labels=True)
    safe_show(_tot, "Topics over time")
else:
    print("Topics over time skipped: some rally dates could not be parsed.")
```

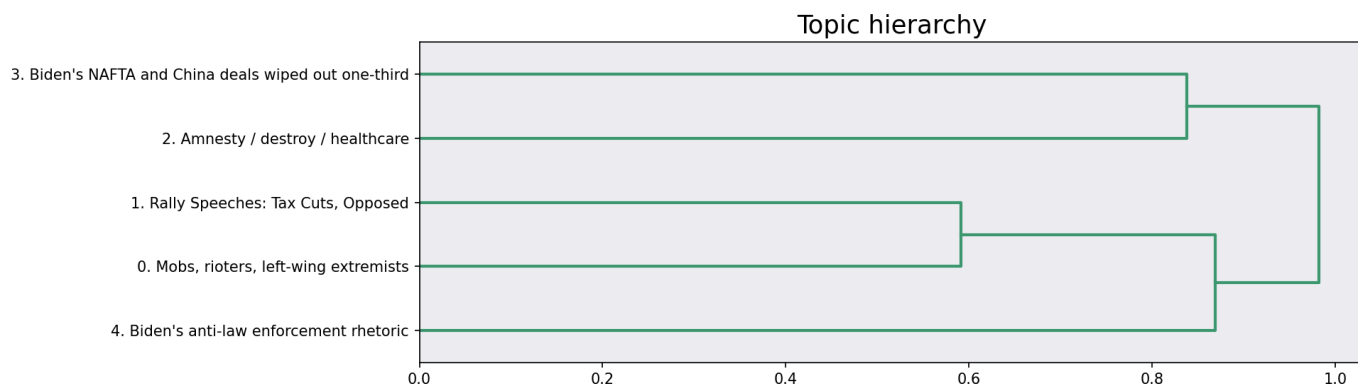


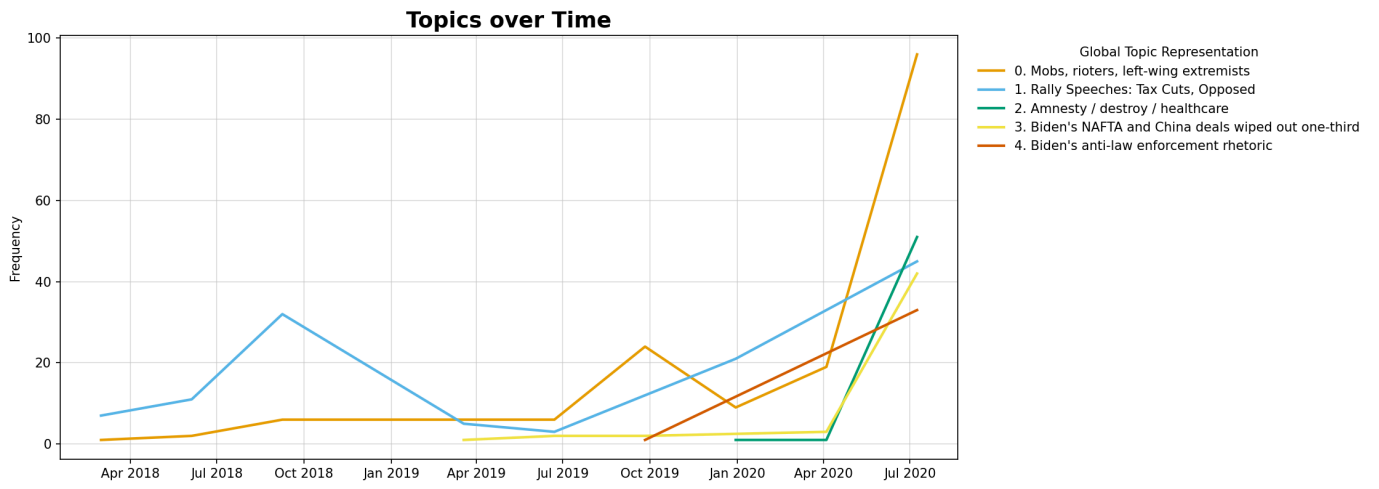
Static render of the interactive Plotly chart

Biden sentences by topic (hover to read)



Static render of the interactive Plotly chart





Static render of the interactive Plotly chart